

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

RAG介绍

- 理解什么是RAG
- 理解RAG解决什么问题
- 理解RAG的工作流程

RAG介绍

通用的基础大模型存在一些问题：

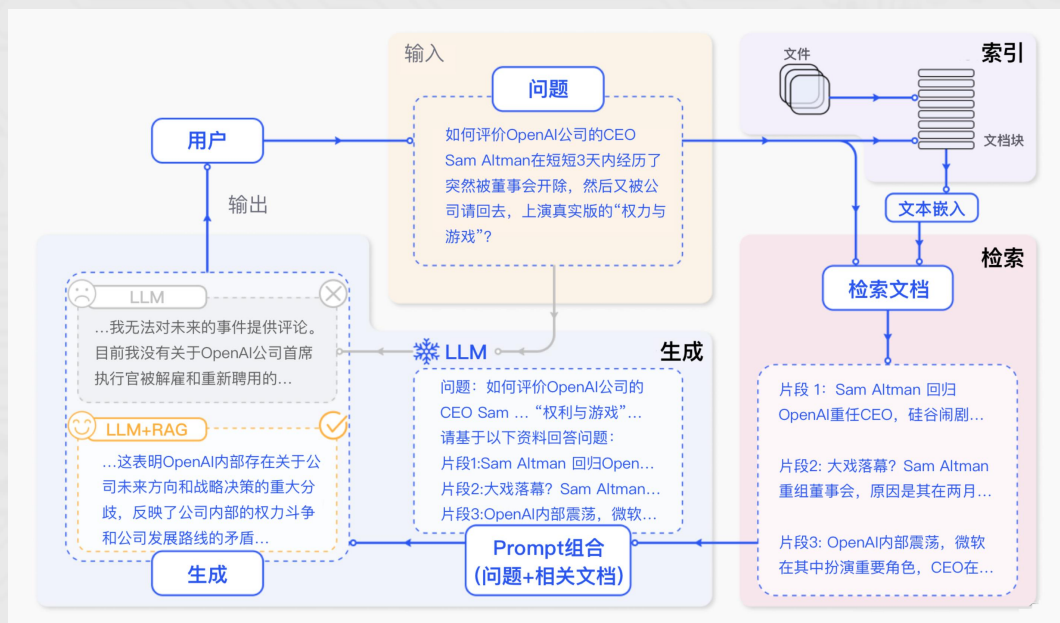
- LLM的知识不是实时的，模型训练好后不具备自动更新知识的能力，会导致部分信息滞后
- LLM领域知识是缺乏的，大模型的知识来源于训练数据，这些数据主要来自公开的互联网和开源数据集，无法覆盖特定领域或高度专业化的内部知识
- 幻觉问题，LLM有时会在回答中生成看似合理但实际上是错误的信息
- 数据安全性



RAG (Retrieval-Augmented Generation) 即检索增强生成，为大模型提供了**从特定数据源检索到的信息，以此来修正和补充生成的答案。**可以总结为一个公式：RAG = 检索技术 + LLM 提示

RAG 的工作原理

工作流程图解：

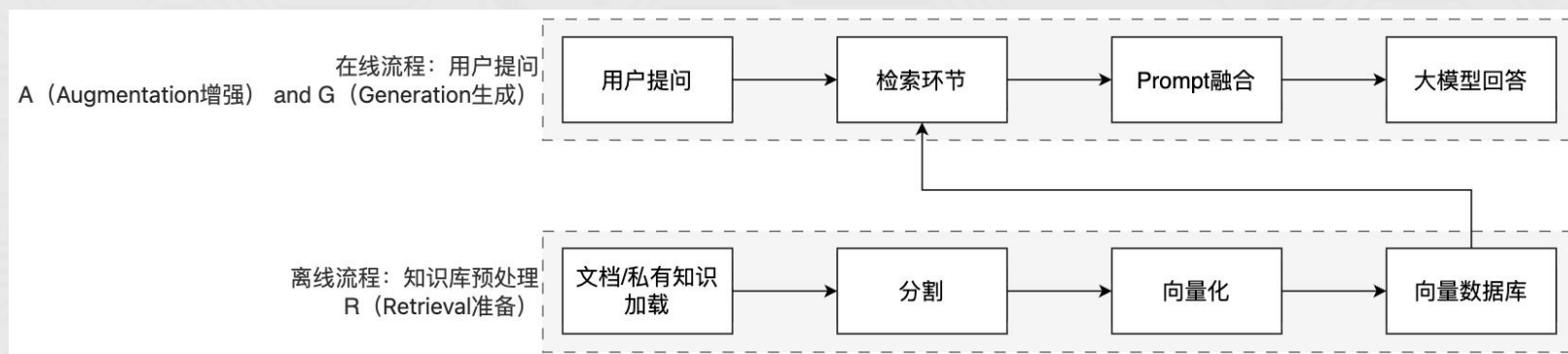


RAG标准流程：



RAG 的工作原理

简单来说，RAG工作分为两条线：
离线准备线 / 在线服务线



RAG标准流程

RAG 标准流程由索引（Indexing）、检索（Retriever）和生成（Generation）三个核心阶段组成。

- **索引阶段**，通过处理多种来源多种格式的文档提取其中文本，将其切分为标准长度的文本块（chunk），并进行嵌入向量化（embedding），向量存储在向量数据库（vector database）中。

- 加载文件
- 内容提取
- 文本分割，形成chunk
- 文本向量化
- 存向量数据库

- **检索阶段**，用户输入的查询（query）被切分为向量块。

- query向量化
- 在文本向量中匹配出与问句向量相似

- **生成阶段**，检索到的相关文本与原始查询

- 匹配出的文本作为上下文和问题一起
- 提交给LLM生成答案：

数据库名称	开源/商业	主要语言	核心特点	部署方式	混合搜索	适用场景
Weaviate	开源	Go	AI原生，多模态支持	云/Docker/K8s/嵌入式	支持	复杂RAG应用
Qdrant	开源	Rust	高性能，稀疏向量支持	云/Docker	支持	大规模向量搜索
Milvus	开源	Go/C++	专为向量搜索优化	云/Docker/K8s	支持	大规模AI应用
MyScale	商业	-	SQL兼容，高性能	云/自托管	支持	SQL开发者
pgvector	开源	C	PostgreSQL扩展	任何PostgreSQL环境	支持	PostgreSQL用户
pgvecto-rs	开源	Rust	高维向量支持	PostgreSQL扩展	支持	需要高维搜索
Chroma	开源	Python	轻量级，易用	云/本地	支持	快速原型开发
OpenSearch	开源	Java	搜索分析一体化	云/自托管	支持	企业搜索应用
TiDB Vector	商业	Go	统一数据管理	云Serverless/专享	支持	分布式AI应用
Elasticsearch	商业	Java	成熟搜索引擎	云/自托管	支持	企业级搜索

的文本块。

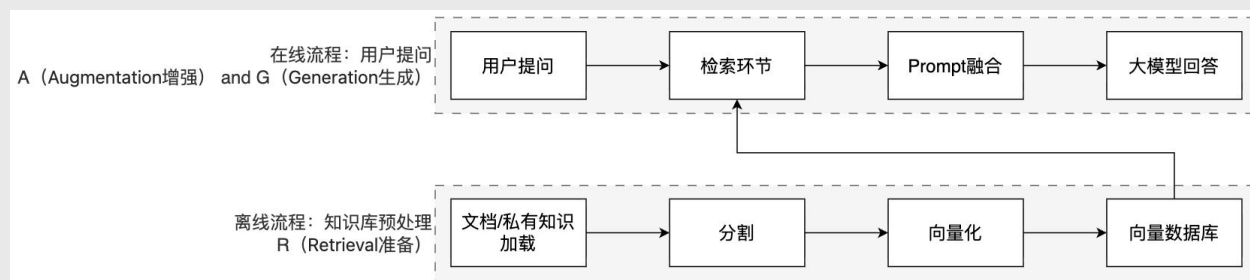
精确且具备上下文关联的回答。

模型本质上就是用户输入，模型给出输出，用户能做的就是输入上做功夫。

RAG就是在向模型提问之前基于已有的知识库或文档内容做检索，确保向模型提问的内容更精准以及包含足够的信息量用以提供给模型。

总结

RAG的核心工作是2个流程：



RAG的核心价值：

- **解决知识时效性问题：**大模型的训练数据有截止时间，RAG 可以接入最新文档（如公司财报、政策文件），让模型输出“与时俱进”。
- **降低模型幻觉：**模型的回答基于检索到的事实性资料，而非纯靠自身记忆，大幅减少编造信息的概率。
- **无需重新训练模型：**相比微调（Fine-tuning），RAG 只需更新知识库，成本更低、效率更高。

AI 大模型

RAG与智能体开发

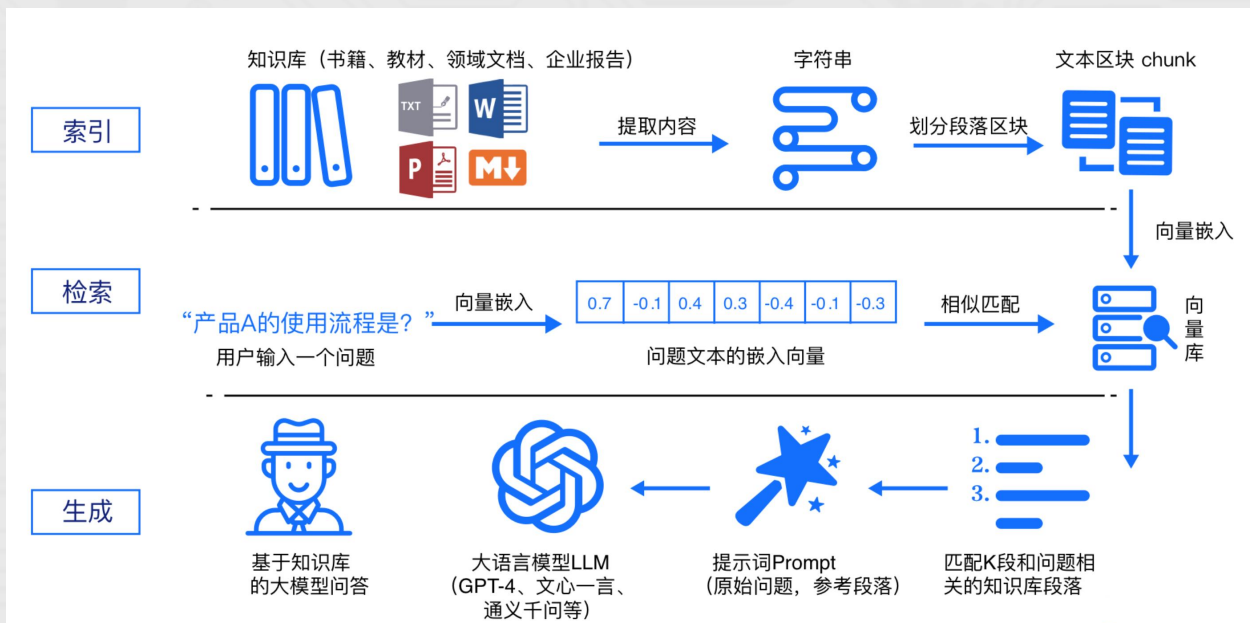
Part : LangChain RAG开发

向量的基础概念

向量的基础概念

RAG流程中，向量库是一个重要的节点。

- 离线流程：知识和信息 → 向量嵌入（向量化） → 存入向量库
- 在线流程：用户的提问 → 向量嵌入（向量化） → 在向量库中匹配



向量是什么？



向量的基础概念

向量 (Vector) 就是文本的“数学身份证”：它把一段文字的**语义信息**，转换成一串固定长度的**数字列表**，让计算机能“看懂”**文字的含义并做相似度计算**。

简单来说，就是让计算机更方便的理解不同的文本内容，是否表述的是一个意思。

如何快速学习RAG

RAG如何快速学会

[1, 3, 6, 2, 1, 7, 9, 10]

[2, 4, 5, 1, 2, 6, 10, 9]

数字随便填的仅表述概念，非真实向量



一个意思



我母鸡呀



什么玩意



一个意思



向量的基础概念

[1, 3, 6, 2, 1, 7, 9, 10]

[2, 4, 5, 1, 2, 6, 10, 9]

文本嵌入模型（如text-embedding-v1）通过深度学习等技术，从文本提取语义特征并映射为固定长度的数字序列。

向量嵌入的过程，我们一般选用合适的文本嵌入模型来完成。

在向量匹配的过程中，如何识别2段文本是否表述相似的含义，主要可以通过如余弦相似度等算法来完成。

比如（下列案例中向量为示例，仅描述概念，非真实向量）：

- A: “如何快速学打篮球” → [0.2, 0.5, 0.8]
- B: “打篮球怎么学得快” → [0.18, 0.52, 0.79]
- C: “运动后吃什么好呢” → [0.9, 0.1, 0.2]

通过余弦相似度算法可以计算得到：A和B相似度0.999789，A和C相似度0.361446

由此可通过精确的数学计算，去匹配2段文本是否描述同一个意思，提高语义匹配的效率 and 精度。



向量的基础概念

如何更为精准的完成语义匹配，生成向量的维度是一个很重要的指标。

如text-embedding-v1模型，可以生成1536维的向量（一段文本**固定**得到1536个数字序列），比较实用。

- 1536个数字表示，这段文本在1536个主题（抽象的语义特征）方向上的得分（强度）

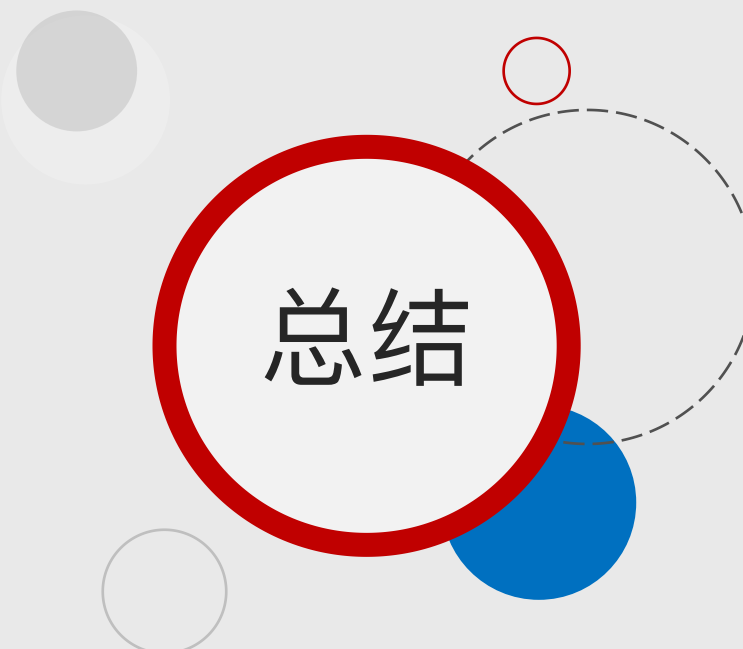


- 生成向量的维度越多，就更好的记录文本的语义特征，做语义匹配会更加精准。
- 更多的向量会在计算、存储和匹配过程中，带来更大的压力。

选择合适的向量维度需要在精确和性能之间做平衡。

一般1536维算是比较好的选择。





总结

向量 (Vector) 就是文本的“数学身份证”

它把一段文字的语义信息，转换成一串固定长度的数字列表，让计算机能“看懂”文字的含义并做相似度计算。

- 向量的计算（文本嵌入过程），可借助文本嵌入模型实现，如text-embedding-v1
- 向量的匹配通过算法实现，如余弦相似度
- 向量的维度表示一段文本在多个抽象语义特征方面的强度
 - 维度数代表模型用多少个抽象语义特征来描述文本
 - 维度越多，做语义匹配越精准
 - 但性能压力也会增大

AI 大模型

RAG与智能体开发

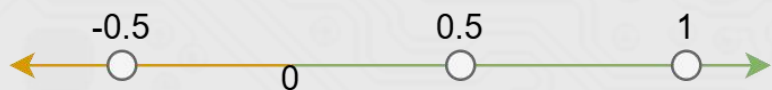
Part : LangChain RAG开发

【扩展】余弦相似度

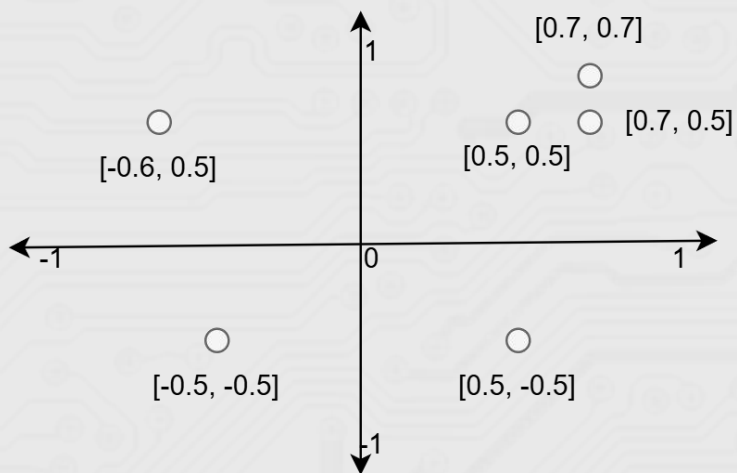
【扩展】余弦相似度

向量的数字序列，共同决定了向量在高维空间中的**方向**和**长度**。而余弦相似度主要就是撇除长度的影响，得到方向的夹角。夹角越小越相似，即方向相同。

如何体现向量的方向和长度呢？以一维向量为例：



向量：[-0.5]、[0.5]、[1]的方向和长度。



向量：
[0.5, 0.5]
[0.7, 0.7]
[0.7, 0.5]
[0.5, -0.5]
[-0.5, -0.5]
[-0.6, 0.5]
的方向和长度

余弦相似度主要匹配的就是：**同向**（无所谓长度）

我们能直接发现

- [0.5, 0.5]和[0.7, 0.7]是同向不同长
那计算机如何判定就依赖余弦相似度算法了。

PS：3维乃至更高纬度难以描述，但概念一致



【扩展】余弦相似度

在文本向量语义匹配中，余弦相似度是衡量两个向量方向相似程度的核心算法，即判断两段文本语义是否相近。

余弦相似度 → 两个向量的点积 ÷ 两个向量模长的乘积 $\text{cosine similarity}(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$

以：A[0.5, 0.5]、B[0.7, 0.7]、C[0.7, 0.5]、D[-0.6, -0.5]为例

- 点积：两个向量在同维度的乘积之和。
 - 向量AB点积： $\text{vec_a}[0] \times \text{vec_b}[0] + \text{vec_a}[1] \times \text{vec_b}[1] + \dots + \text{vec_a}[n] \times \text{vec_b}[n]$
 - 如AB的点积是： $0.5 \times 0.7 + 0.5 \times 0.7 = 0.74$
- 模长：单个向量不同维度的平方之和开根号，如A的模长是： $\sqrt{0.5 \times 0.5 + 0.5 \times 0.5}$
 - 向量模长： $\|\text{vec}\| = \sqrt{(\text{vec}[0])^2 + (\text{vec}[1])^2 + \dots + (\text{vec}[n])^2}$
 - 如向量A的模长： $\sqrt{0.5 \times 0.5 + 0.5 \times 0.5}$ $\sqrt{\quad}$ 是开根号

如AB之间的余弦相似度为：AB点积 ÷ (A模长 * B模长)

- AB余弦相似度： $(0.5 \times 0.7 + 0.5 \times 0.7) \div (\sqrt{0.5 \times 0.5 + 0.5 \times 0.5} * \sqrt{0.7 \times 0.7 + 0.7 \times 0.7}) = 1.0$
- AC 余弦相似度： $(0.5 \times 0.7 + 0.5 \times 0.5) \div (\sqrt{0.5 \times 0.5 + 0.5 \times 0.5} * \sqrt{0.7 \times 0.7 + 0.5 \times 0.5}) \approx 0.986$
- AD 余弦相似度： $(0.5 \times (-0.6) + 0.5 \times (-0.5)) \div (\sqrt{0.5 \times 0.5 + 0.5 \times 0.5} * \sqrt{(-0.6) \times (-0.6) + (-0.5) \times (-0.5)}) \approx -0.996$



AI 大模型

RAG与智能体开发

Part : LangChain RAG开发



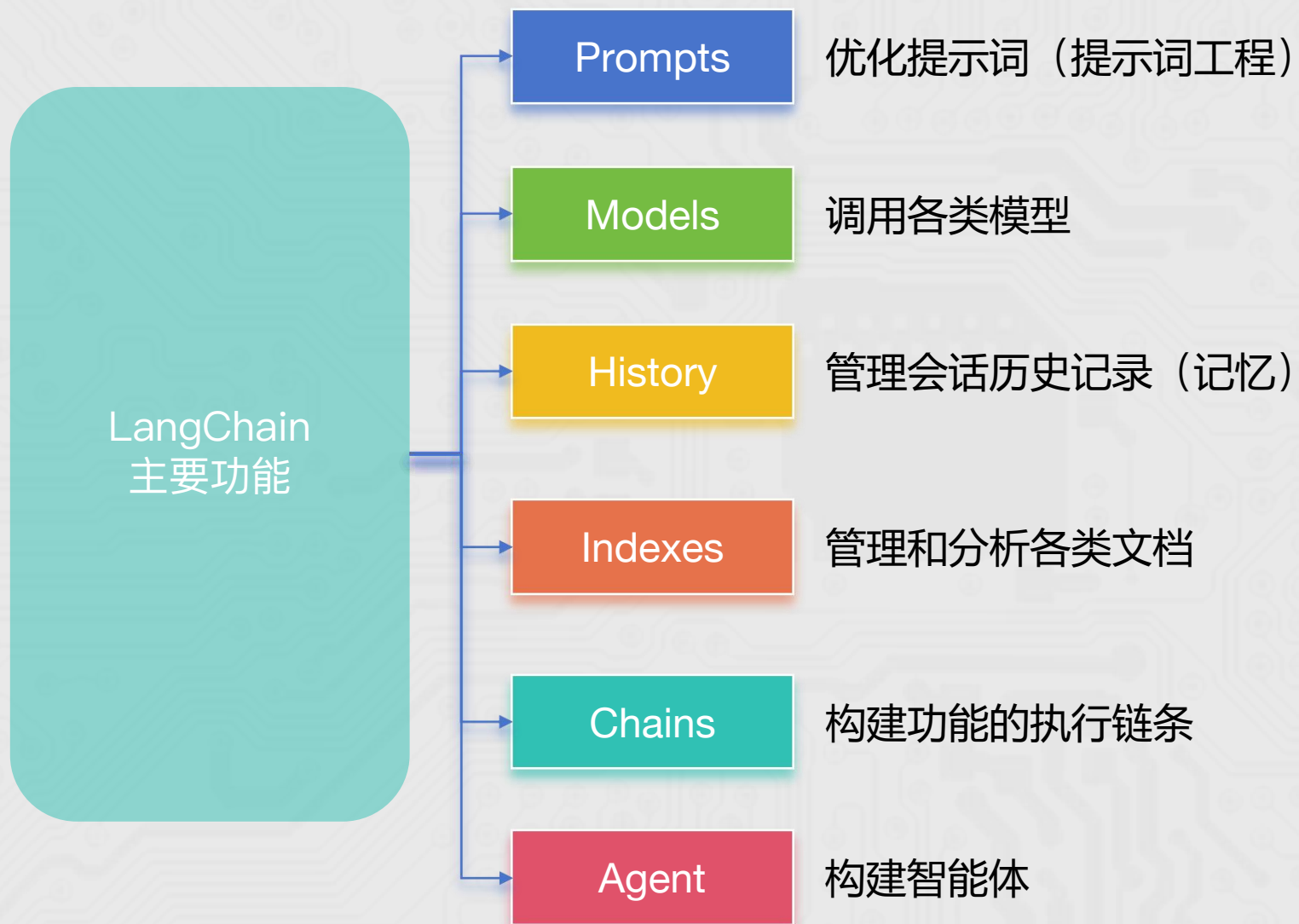
LangChain



LangChain 由 Harrison Chase 创建于2022年10月，它是围绕LLMs（大语言模型）建立的一个框架。



LangChain自身并不开发LLMs，它的核心理念是**为各种LLMs实现通用的接口**，把LLMs相关的组件“链接”在一起，简化LLMs应用的开发难度，方便开发者快速地开发复杂的LLMs应用。



All in LangChain
一站齐活

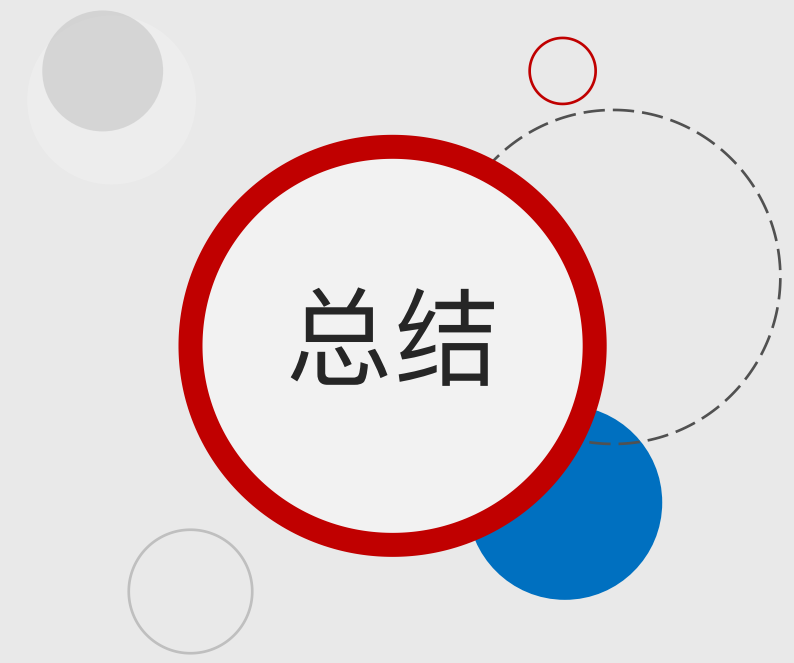
LangChain是一个开发LLM相关业务功能的集大成者，
是一个Python的第三方库，提供了各种功能的API。

提供：

- 提示词优化的相关功能API
- 调用各类模型的功能API
- 会话记忆的相关功能API
- 各类文档管理分析的功能API
- 构建Agent智能体的相关功能API
- 各类功能链式执行的能力

LangChain是后续学习RAG开发的主力框架

Full LLM power—you only need LangChain



总结

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

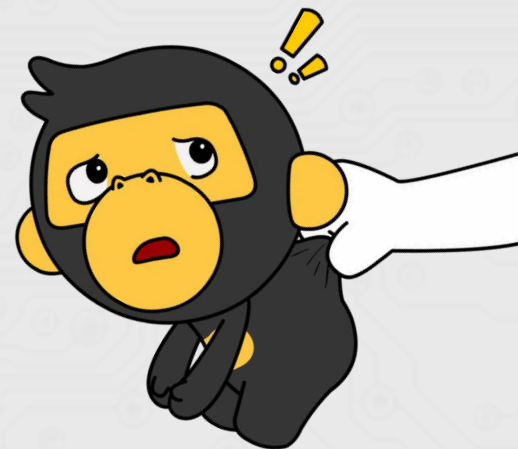


LangChain 环境部署



```
pip install langchain langchain-community langchain-ollama langchain-chroma dashscope chromadb bs4 jq
```

- langchain: 核心包
- langchain-community: 社区支持包，提供了更多的第三方模型调用（我们用的阿里云千问模型就需要这个包）
- langchain-ollama: Ollama支持包，支持调用Ollama托管部署的本地模型
- langchain-chroma: ChromaDB支持包，支持调用ChromaDB
- dashscope: 阿里云通义千问的Python SDK
- chromadb: 轻量向量数据库（后续使用）
- bs4: BeautifulSoup4库，协助解析HTML文档（后续学习文档加载器使用）



AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

Models: 大语言模型的使用

Models

现在市面上的模型多如牛毛，各种各样的模型不断出现，LangChain模型组件提供了与各种模型的集成，并为所有模型提供一个精简的统一接口。

LangChain目前支持三种类型的模型：LLMs（大语言模型）、Chat Models(聊天模型)、Embeddings Models(嵌入模型)。

- LLMs:是技术范畴的统称，指基于大参数量、海量文本训练的 Transformer 架构模型，核心能力是理解和生成自然语言，主要服务于文本生成场景
- 聊天模型:是应用范畴的细分，是专为对话场景优化的 LLMs，核心能力是模拟人类对话的轮次交互，主要服务于聊天场景
- 文本嵌入模型: 文本嵌入模型接收文本作为输入，得到文本的向量。

LangChain支持的三类模型，它们的使用场景不同，输入和输出不同，开发者需要根据项目需要选择相应。

我们所用的阿里云通义千问系列主要来自于：langchain_community包



LLMs（阿里云大语言模型的访问）

LLMs使用场景最多，常用大模型的下载库：

- <https://huggingface.co/models>
- <https://modelscope.cn/models>

同时LangChain支持对许多模型的调用，以通义千问为例：

```
from langchain_community.llms.tongyi import Tongyi
### 实例化模型
llm = Tongyi(model='qwen-max')
### 模型推理
res = llm.invoke("帮我讲个笑话吧")
print(res)
```

```
D:\miniconda3\python.exe E:\work\czbk\大模型技术开发与应用-备课\03-code\day05\lc_models2.py
```

当然可以，接下来是一个轻松的笑话：

为什么电脑经常生病？

因为它的窗户（Windows）总是开着！



LLMs (Ollama本地大语言模型的访问)

如果要访问本地Ollama的模型，简单更改一下代码。

通过langchain_ollama包导入OllamaLLM类即可（请确保Ollama已经启动并提前下载好要使用的模型）。

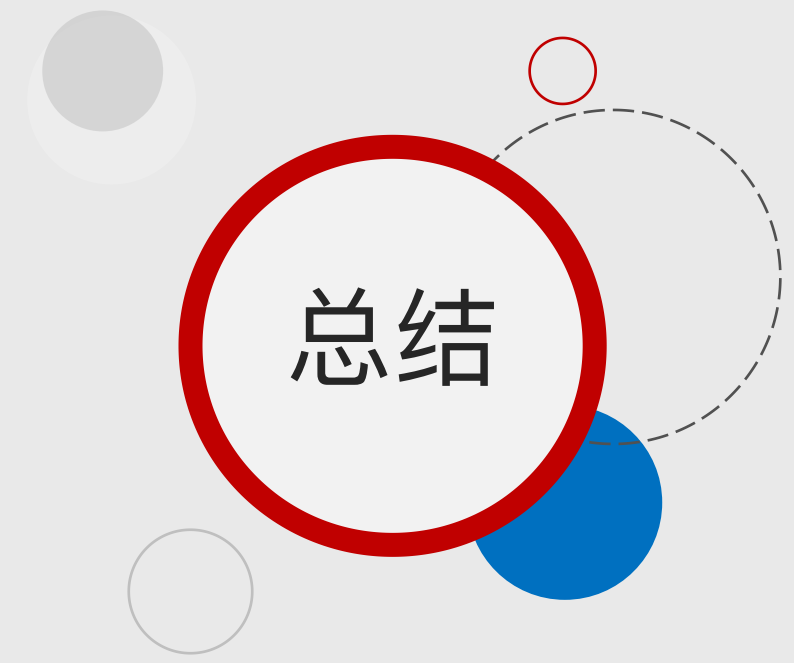
```
from langchain_ollama import OllamaLLM

model = OllamaLLM(model="qwen3:4b")

# 通过invoke方法去调用模型
res = model.invoke(input="你是谁呀能做什么? ")

print(res)
```





总结

通过：

- `from langchain_community.llm.tongyi import Tongyi` 导入通义千问系列的支持
- `from langchain_ollama import OllamaLLM` 导入 Ollama 系列的支持

创建好模型对象后，通过 `invoke` 对模型发起提问
并可以直接打印输出结果

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

Models的流式输出

模型的流式输出

如果需要流式输出结果，需要将模型的invoke方法改为stream方法即可。

- invoke方法：一次型返回完整结果
- stream方法：逐段返回结果，流式输出

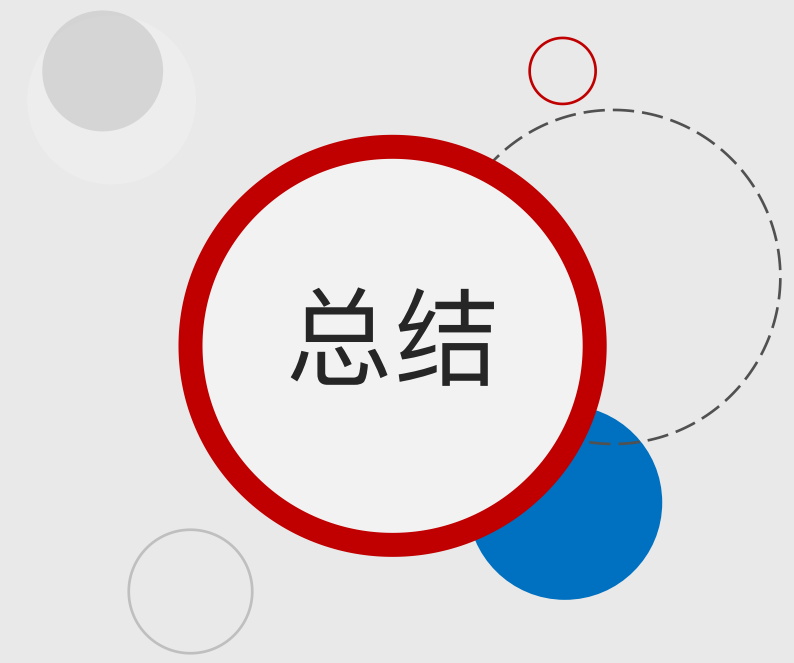
```
# langchain_community
from langchain_community.llms.tongyi import Tongyi

# 不用qwen3-max, 因为qwen3-max是聊天模型, qwen-max是大语言模型
model = Tongyi(model="qwen-max")

# 调用invoke向模型提问
res = model.stream(input="你是谁呀能做什么? ")

for chunk in res:
    print(chunk, end="", flush=True)
```





总结

模型对象有2个方法去调用模型：

- `invoke`，调用模型，一次型返回完整结果
- `stream`，调用模型，逐段流式输出

这两个方法是新版LangChain(1.0版本后)中基于Runnable接口的通用核心方法。

绝大多数组件（如提示词模板、链、向量检索、工具调用等，后续学习）都支持这两个方法，这也是 LangChain 设计的核心统一范式。

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

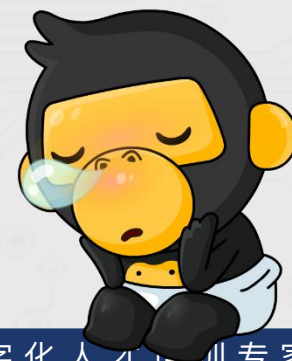
LangChain组件

Models: 聊天模型的使用

Chat Models (聊天模型)

聊天消息包含下面几种类型，使用时需要按照约定传入合适的值：

- `AIMessage`: 就是 AI 输出的消息，可以是针对问题的回答. (OpenAI库中的assistant角色)
- `HumanMessage`: 人类消息就是用户信息，由人给出的信息发送给LLMs的提示信息，比如“实现一个快速排序方法”. (OpenAI库中的user角色)
- `SystemMessage`: 可以用于指定模型具体所处的环境和背景，如角色扮演等。你可以在这里给出具体的指示，比如“作为一个代码专家”，或者“返回json格式”. (OpenAI库中的system角色)



Chat Models (聊天模型)

演示单独使用HumanMessage

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.messages import HumanMessage

# 初始化模型
chat = ChatTongyi(model="qwen3-max")

# 准备消息list
messages = [
    HumanMessage(content="给我写一首唐诗")
]

# 流式输出
for chunk in chat.stream(input=messages):
    print(chunk.content, end="", flush=True)
```



```
/Users/caoyu/miniconda3/envs/langchain/bin/python /Users
好的，这是一首为您创作的七言绝句，力求体现唐诗的意境、格律与韵味：

---

**《秋江晚泊》**
孤舟泊处暮烟平，
雁字横天霜月清。
芦荻萧萧风满袖，
一灯如豆照离情。

---
```



Chat Models (聊天模型)

演示SystemMessage + HumanMessage

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.messages import HumanMessage, SystemMessage
```

初始化模型

```
chat = ChatTongyi(model="qwen3-max")
```

准备消息list

```
messages = [
    SystemMessage(content="你是一名来自边塞的诗人"),
    HumanMessage(content="给我写一首唐诗")
]
```

流式输出

```
for chunk in chat.stream(input=messages):
    print(chunk.content, end="", flush=True)
```

```
/Users/caoyu/miniconda3/envs/langchain/bin/python /Users/caoyu/D
*抬头望向远方的烽火台，眼神中透着几分沧桑，缓缓吟诵道*  
《塞上曲》  
  
黄沙百战铁衣寒，  
孤城遥望玉门关。  
胡笳声断月如霜，  
一夜征人尽望乡。  
  
大漠风急卷旌旗，  
血染残阳照马蹄。  
忽见天边雁字回，  
将军白发泪沾衣。  
  
*轻叹一声，手指无意识地摩挲着腰间的酒壶*  
  
这苦战过的边塞多年所见所感，每当夜深人静，听着远处传来的胡笳声，
```



Chat Models (聊天模型)

演示SystemMessage + HumanMessage + AIMessage

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.messages import HumanMessage, SystemMessage, AIMessage

# 初始化模型
chat = ChatTongyi(model="qwen3-max")

# 准备消息list
messages = [
    SystemMessage(content="你是一名来自边塞的诗人"),
    HumanMessage(content="给我写一首唐诗"),
    AIMessage(content="锄禾日当午，汗滴禾下土，谁知盘中餐，粒粒皆辛苦。"),
    HumanMessage(content="给予你上一首的格式，再来一首")
]

# 流式输出
for chunk in chat.stream(input=messages):
    print(chunk.content, end="", flush=True)
```



```
/Users/caoyu/minicond
边城月似钩，霜刃照寒秋。
黄沙埋战骨，孤雁唳荒丘。
烽火连三载，家书抵万金。
何当饮胡血，归卧故园阴？
进程已结束，退出代码为 0
```



AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

Models: 消息的简写形式

消息的简写形式

SystemMessage、HumanMessage、AIMessage可以有如下的简写形式

```
# 得到模型对象, qwen3-max就是聊天模型
model = ChatTongyi(model="qwen3-max")

# 准备消息列表
messages = [
    SystemMessage(content="你是一个边塞诗人。"),
    HumanMessage(content="写一首唐诗"),
    AIMessage(content="锄禾日当午, 汗滴禾下土, 谁知盘中餐, 粒粒皆辛苦。"),
    HumanMessage(content="按照你上一个回复的格式, 在写一首唐诗。")
]

# for循环迭代打印输出, 通过.content来获取到内容
for chunk in model.stream(input=messages):
    print(chunk.content, end="", flush=True)
```

```
# 得到模型对象, qwen3-max就是聊天模型
model = ChatTongyi(model="qwen3-max")

# 准备消息列表
messages = [
    ("system", "你是一个边塞诗人。"),
    ("human", "写一首唐诗"),
    ("ai", "锄禾日当午, 汗滴禾下土, 谁知盘中餐, 粒粒皆辛苦。"),
    ("human", "按照你上一个回复的格式, 再写一首唐诗。")
]

# for循环迭代打印输出, 通过.content来获取到内容
for chunk in model.stream(input=messages):
    print(chunk.content, end="", flush=True)
```



通过2元元组封装信息;

- 第一个元素为角色
 - 字符串: system/human/ai
- 第二个元素为内容



消息的简写形式

区别和优势在于，使用类对象的方式，如下：

```
messages = [  
    SystemMessage(content="内容..."),  
    HumanMessage(content="内容..."),  
    AIMessage(content="内容..."),  
]
```

是**静态**的，一步到位
直接就得到了Message类的类对象

简写形式如下：

```
messages = [  
    ("system", "内容..."),  
    ("human", "内容..."),  
    ("ai", "内容..."),  
]
```

是**动态**的，需要在**运行时**
由LangChain内部机制**转换**为Message类对象

好处就在于，简写形式避免导包、写起来更简单，更重要的是支持：

```
messages = [  
    ("system", "今天的天气是{weather}"),  
    ("human", "我的名字是：{name}"),  
    ("ai", "欢迎{lastname}先生"),  
]
```

由于是动态，需要转换步骤
所以简写形式支持内部填充{变量}占位
可在运行时填充具体值(后续学习提示词模板时用到)



AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

Models: 嵌入模型的使用

Embeddings Models(文本嵌入模型)

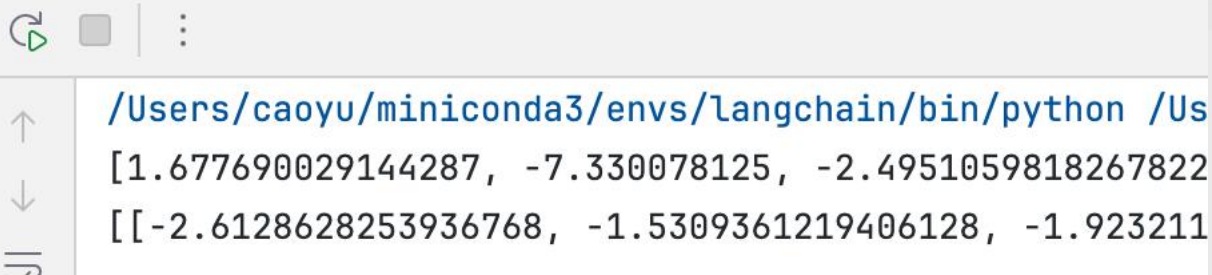
Embeddings Models嵌入模型的特点：将字符串作为输入，返回一个浮点数的列表（向量）。
在NLP中，Embedding的作用就是将数据进行文本向量化。

阿里云千问模型访问方式：

```
from langchain_community.embeddings import DashScopeEmbeddings

# 初始化嵌入模型对象，其默认使用模型是：text-embedding-v1
embed = DashScopeEmbeddings()

# 测试
print(embed.embed_query("我喜欢你"))
print(embed.embed_documents(['我喜欢你', '我稀饭你', '晚上吃啥']))
```



```
/Users/caoyu/miniconda3/envs/langchain/bin/python /Us
[1.677690029144287, -7.330078125, -2.4951059818267822
[[-2.6128628253936768, -1.5309361219406128, -1.923211
```



Embeddings Models(文本嵌入模型)

本地Ollama模型访问方式:

通过langchain_ollama导入OllamaEmbeddings使用, 其余不变。

```
from langchain_ollama import OllamaEmbeddings
# 初始化嵌入模型对象, 其默认使用模型是: text-embedding-v1
embed = OllamaEmbeddings(model="qwen3-embedding")
# 测试
print(embed.embed_query("我喜欢你"))
print(embed.embed_documents(['我喜欢你', '我稀饭你', '晚上吃啥']))
```

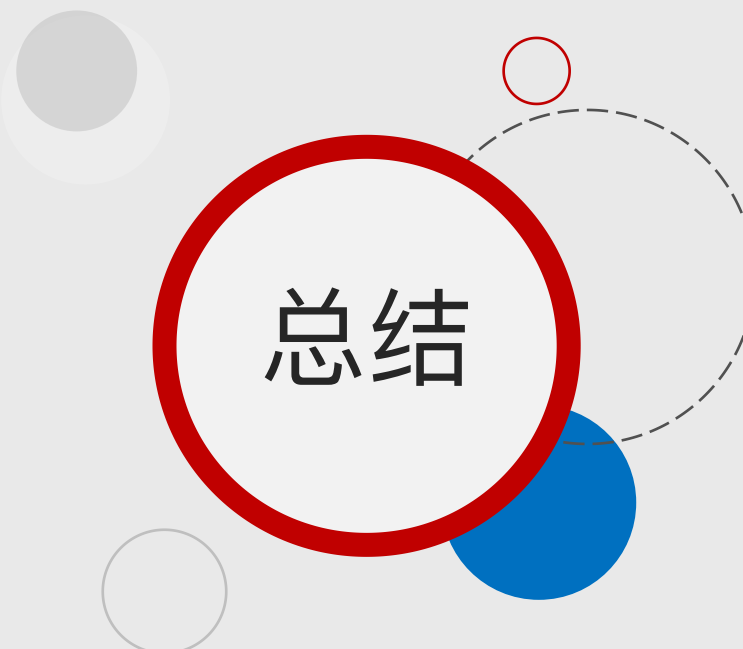


模型使用总结

目前所掌握的LangChain API如下：

方式	LLMs大语言模型	聊天模型	文本嵌入模型
阿里云千问	<pre>from langchain_community.llms.tongyi import Tongyi</pre>	<pre>from langchain_community.chat_models.tongyi import ChatTongyi</pre>	<pre>from langchain_community.embeddings import DashScopeEmbeddings</pre>
Ollama本地模型	<pre>from langchain_ollama import OllamaLLM</pre>	<pre>from langchain_ollama import ChatOllama</pre>	<pre>from langchain_ollama import OllamaEmbeddings</pre>
方法	invoke批量 / stream流式	invoke批量 / stream流式	embed_query单次转换 / embed_documents批量转换





总结

LangChain在模型的支持上主要基于LangChain_community包提供。

主要支持三类模型：

- LLMs：大语言模型，主用于文本生成
- Chat Model：聊天模型，主用于多轮次对话的聊天场景
- Embeddings Model：文本嵌入模型，主用于生成文本向量

LangChain框架和OpenAI库一样，提供三种角色：

- HumanMessage类，即User角色
- AIMessage类，即Assistant角色
- SystemMessage类，即System角色

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

通用Prompts

通用prompt(zero-shot)

提示词优化在模型应用中非常重要，LangChain提供了PromptTemplate类，用来协助优化提示词。

PromptTemplate表示提示词模板，可以构建一个自定义的基础提示词模板，支持变量的注入，最终生成所需的提示词。

```
from langchain_core.prompts import PromptTemplate
from langchain_community.llms.tongyi import Tongyi

prompt_template = PromptTemplate.from_template(
    "我的邻居姓{lastname}，刚生了{gender}，帮忙起名字，请简略回答。"
)

# 变量注入，生成提示词文本
prompt_text = prompt_template.format(lastname="张", gender="女儿")

model = Tongyi(model="qwen-max")          # 创建模型对象
res = model.invoke(input=prompt_text)      # 调用模型获取结果
print(res)
```

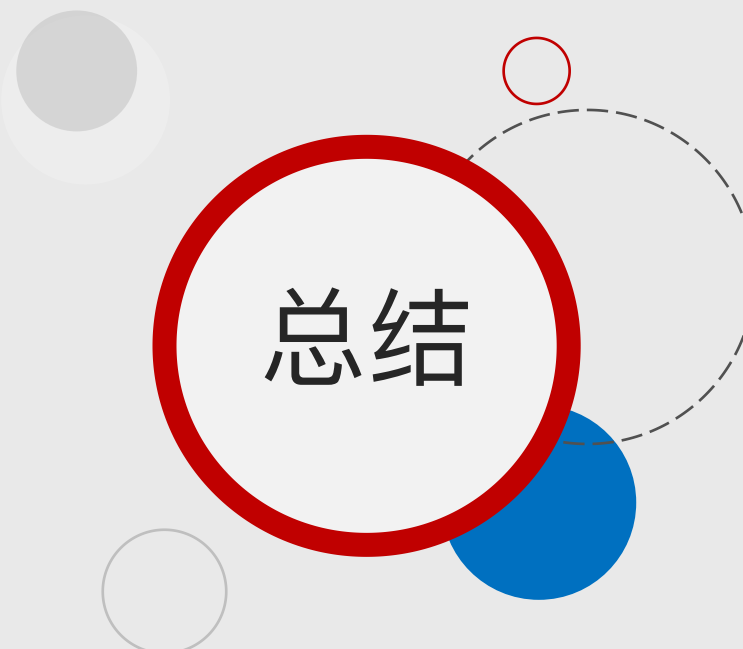
标准写法

```
from langchain_core.prompts import PromptTemplate
from langchain_community.llms.tongyi import Tongyi

prompt_template = PromptTemplate.from_template(
    "我的邻居姓{lastname}，刚生了{gender}，帮忙起名字，请简略回答。"
)

model = Tongyi(model="qwen-max")          # 创建模型对象
# 生成链
chain = prompt_template | model
# 基于链，调用模型获取结果
res = chain.invoke(input={"lastname": "曹", "gender": "女儿"})
print(res)
```

基于chain链的写法



总结

基于PromptTemplate类可以得到提示词模板，支持基于模板注入变量得到最终提示词。

- zero-shot思想下，可以基于PromptTemplate直接完成。
- few-shot思想下，需要更换为FewShotPromptTemplate（后续学习）

PS：使用PromptTemplate还不如自己手动拼接字符串？

- 使用Template模板构建提示词，在大型工程中更容易做标准化模板
- Template模板类，支持LangChain框架的链式调用（Runnable接口，后续学习）
 - PromptTemplate
 - FewShotPromptTemplate(后续学习)
 - ChatPromptTemplate(后续学习)

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

FewShotPromptTemplate的使用

FewShotPromptTemplate

```
from langchain_core.prompts import FewShotPromptTemplate
```

```
FewShotPromptTemplate(  
    examples=None,  
    example_prompt=None,  
    prefix=None,  
    suffix=None,  
    input_variables=None  
)
```

参数：

- examples: 示例数据, list, 内套字典
- example_prompt: 示例数据的提示词模板
- prefix: 组装提示词, 示例数据前内容
- suffix: 组装提示词, 示例数据后内容
- input_variables: 列表, 注入的变量列表



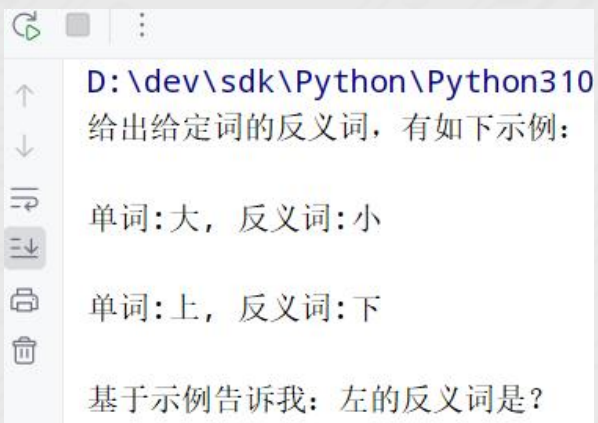
组装FewShotPromptTemplate对象并获得最终提示词

```
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate

example_template = PromptTemplate.from_template("单词:{word}, 反义词:{antonym}")

example_data = [                                     # 示例数据, list内套字典
    {"word": "大", "antonym": "小"},
    {"word": "上", "antonym": "下"}
]

few_shot_prompt = FewShotPromptTemplate(             # FewShot提示词模板对象
    example_prompt=example_template,
    examples=example_data,
    prefix="给出给定词的反义词, 有如下示例: ",
    suffix="基于示例告诉我: {input_word}的反义词是? ",
    input_variables=['input_word']
)
# 获得最终提示词
prompt_text = few_shot_prompt.invoke(input={"input_word": "左"}).to_string()
print(prompt_text)
```



```
D:\dev\sdk\Python\Python310
给出给定词的反义词, 有如下示例:

单词:大, 反义词:小

单词:上, 反义词:下

基于示例告诉我: 左的反义词是?
```



调用模型获得结果

```
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
from langchain_community.chat_models.tongyi import ChatTongyi

example_template = PromptTemplate(    # 示例的提示词模板对象
    input_variables=['word', 'antonym'],
    template="word: {word}, antonym: {antonym}"
)

example_data = [                    # 示例数据, list内套字典
    {"word": "大", "antonym": "小"},
    {"word": "上", "antonym": "下"}
]

few_shot_prompt = FewShotPromptTemplate(# FewShot提示词模板对象
    examples=example_data,
    example_prompt=example_template,
    prefix="给出给定词的反义词, 有如下示例:",
    suffix="基于示例告诉我: {input_word}的反义词是?",
    input_variables=['input_word']
)

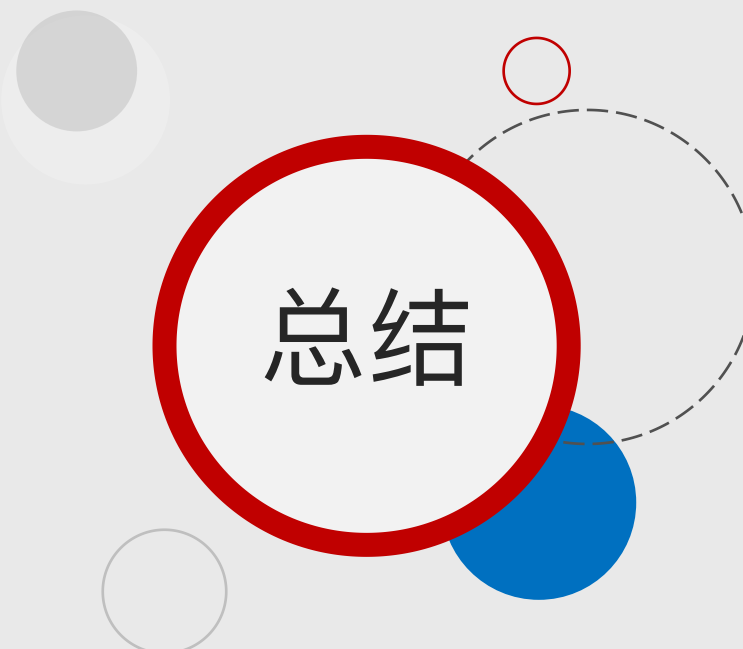
# 获得最终提示词
prompt_text = few_shot_prompt \
    .invoke(input={"input_word": "左"}) \
    .to_string()

model = ChatTongyi(model="qwen3-max")
for chunk in model.stream(input=prompt_text):
    print(chunk.content, end="", flush=True)
```



```
/Users/caoyu/miniconda3/envs
word: 左, antonym: 右
进程已结束, 退出代码为 0
```





总结

FewShotPromptTemplate类对象构建需要5个核心参数：

- example_prompt：示例数据的提示词模板
- examples：示例数据，list，内套字典
- prefix：组装提示词，示例数据前内容
- suffix：组装提示词，示例数据后内容
- input_variables：列表，注入的变量列表

```
few_shot_template = FewShotPromptTemplate(  
    example_prompt=example_template,      # 示例数据的模板  
    examples=examples_data,              # 示例的数据（用来注入动态数据的），list内套字典  
    prefix="告知我单词的反义词，我提供如下的示例：",      # 示例之前的提示词  
    suffix="基于前面的示例告知我，{input_word}的反义词是？",      # 示例之后的提示词  
    input_variables=['input_word']      # 声明在前缀或后缀中所需要注入的变量名  
)  
  
prompt_text = few_shot_template.invoke(input={"input_word": "左"}).to_string()  
print(prompt_text)
```



练习

FewShotPromptTemplate练习案例

在前面学习few-shot思想的时候，我们有一个文字案例。
请基于代码，构建FewShotPromptTemplate，并调用模型获得结果。

请抽取产品名称和核心卖点2个字段，格式为Json，我提供2个示例。

• 示例1: MacBookPro高效节能，性能强大，适合牛马工作使用

输出: {"产品名称": "MacBookPro", "产品卖点": "高效节能，性能强大"}

• 示例2: 联想笔记本拥有RTX4060独立显卡，畅玩游戏，丝滑流畅

输出: {"产品名称": "联想笔记本", "产品卖点": "畅玩游戏，丝滑流畅"}

请处理: 华为MatepadPro，高清大屏，长效续航，你的好帮手。

{"产品名称": "华为MatepadPro", "产品卖点": "高清大屏，长效续航"}



AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

模板类的format和invoke方法

模板类的format和invoke方法

在PromptTemplate（通用提示词模板）和FewShotPromptTemplate（FewShot提示词模板）的使用中，我们使用了如下：

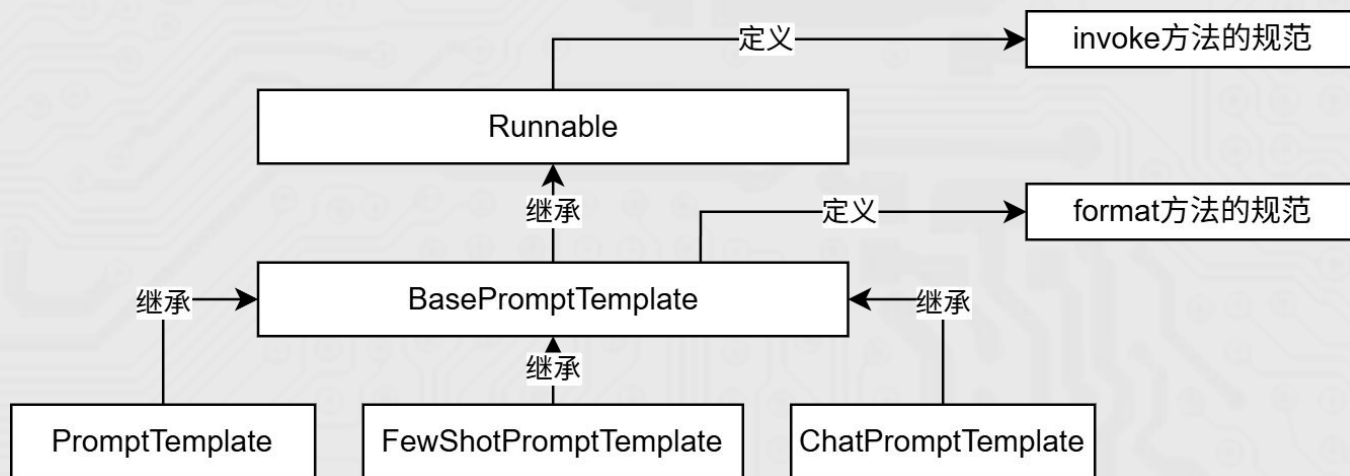
```
7 prompt_template = PromptTemplate.from_template(
8     "我的邻居姓{lastname}, 刚生了{gender}, 你帮我起个名字, 简单回答。"
9 )
10 # 调用.format方法注入信息即可
11 prompt_text = prompt_template.format(lastname="张", gender="女儿")
12
13 model = Tongyi(model="qwen-max")
14 res = model.invoke(input=prompt_text)
15 print(res)
```

模板对象的format方法

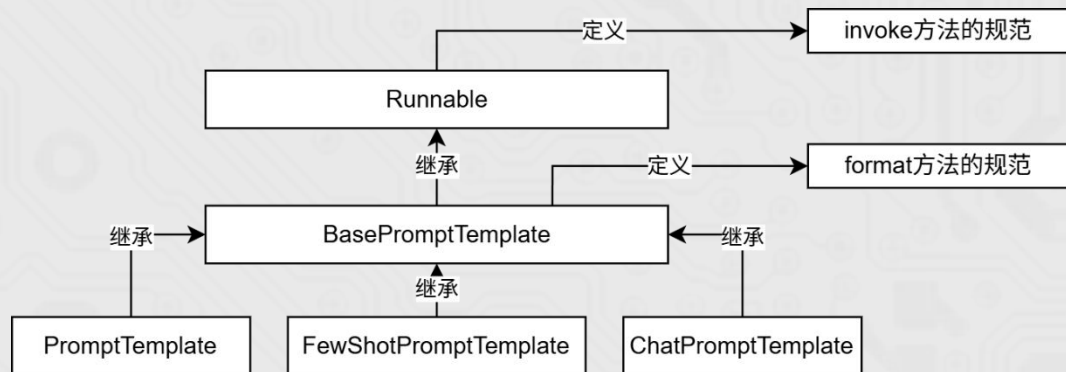
```
13 few_shot_template = FewShotPromptTemplate(
14     example_prompt=example_template, # 示例数据的模板
15     examples=examples_data, # 示例的数据（用来注入动态数据的），list内套
16     prefix="告知我单词的反义词, 我提供如下的示例: ", # 示例之前的提示
17     suffix="基于前面的示例告知我, {input_word}的反义词是? ", # 示例之后的提示
18     input_variables=['input_word'] # 声明在前缀或后缀中所需要注入的变量名
19 )
20
21 prompt_text = few_shot_template.invoke(input={"input_word": "左"}).to_string()
22 print(prompt_text)
```

模板对象的invoke方法

PromptTemplate、FewShotPromptTemplate、ChatPromptTemplate（后续学习）都拥有format和invoke这2类方法。



模板类的format和invoke方法



format和invoke的区别在于：

区别	format	invoke
功能	纯字符串替换，解析占位符生成提示词	Runnable 接口标准方法，解析占位符生成提示词
返回值	字符串	PromptValue类对象
传参	<code>.format(k=v, k=v, ...)</code>	<code>.invoke({"k":v, "k":v, ...})</code>
解析	支持解析 {} 占位符	支持解析 {} 占位符和MessagesPlaceholder结构化占位符

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

ChatPromptTemplate的使用

ChatPromptTemplate

PromptTemplate: 通用提示词模板, 支持动态注入信息。

FewShotPromptTemplate: 支持基于模板注入任意数量的示例信息。

ChatPromptTemplate: 支持注入任意数量的**历史会话**信息。

- 通过from_messages方法, 从列表中获取多轮次会话作为聊天的基础模板
 - PS: 前面PromptTemplate类用的from_template仅能接入一条消息, 而from_messages可以接入一个list的消息

```
from langchain_core.prompts import ChatPromptTemplate

ChatPromptTemplate.from_messages(
    [
        ("system", "....."),
        ("ai", "....."),
        .....,
        ("human", ".....")
    ]
)
```



好像没有信息需要注入呀。
那我用提示词模板类干啥?

ChatPromptTemplate

历史会话信息并不是静态的（固定的），而是随着对话的进行不停地积攒，即动态的。
所以，历史会话信息需要支持动态注入。

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.prompts import MessagesPlaceholder

chat_template = ChatPromptTemplate.from_messages(
    [
        ("system", "....."),
        ("ai", "....."),
        MessagesPlaceholder("history"),
        ("human", ".....")
    ]
)

history_data = [
    ("human", "..."),
    ("ai", "..."),
    ("human", "..."),
    ("ai", "...")
]

prompt_value = chat_template.invoke({"history": history_data})
```

- MessagePlaceholder作为占位
 - 提供history作为占位的key
- 基于invoke动态注入历史会话记录

必须是invoke，format无法注入

chatprompts

也可以基于聊天模型，并组装聊天历史的模式，做提示词工程。

few-shot提示方式：求反义词 *examples = [{"word": "开心", "antonym": "难过"}, {"word": "高", "antonym": "矮"},]*

```
from langchain_community.chat_models import ChatTongyi
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "给出每个单词的反义词",
         # 存储多轮对话的历史记录 history是占位符名称,后续从字典按history作为key取value替代内容
         MessagesPlaceholder("history"),
         ("human", "{question}")
        ]
)
model = ChatTongyi(model="qwen3-max")
# StrOutputParser() LangChain内置的结果解析器,可以直接提取结果文本内容,剔除其余元数据信息
chain = prompt | model | StrOutputParser()
# 无历史会话的提问
for chunk in chain.stream(input={"history": [], "question": "粗"}):
    print(chunk)
print("\n\n")
# 带有历史的提问 用 HumanMessage (用户消息) 和 AIMessage (模型消息) 封装历史对话
history = [ # history 要求是一个列表,内部封装用户和AI的对话记录
    HumanMessage(content="开心"), # 对应 ("human", "开心")
    AIMessage(content="难过"), # 对应 ("ai", "难过")
    HumanMessage(content="高"), # 对应 ("human", "高")
    AIMessage(content="矮") # 对应 ("ai", "矮")
]
# 简化写法,元组的第一个元素是角色(标准角色名 human ai) 第二个元素是消息
# history = [
#     ("human", "开心"), ("ai", "难过"),
#     ("human", "高"), ("ai", "矮")
# ]
for chunk in chain.stream(input={"history": history, "question": "粗"}):
    print(chunk)
```

运行 08_prompt_with_chatmodel

↑

↓

↶

↷

🖨

🗑

/opt/homebrew/Caskroom

“

粗”的反义词是

: 细。

细

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

chains 链的基础使用

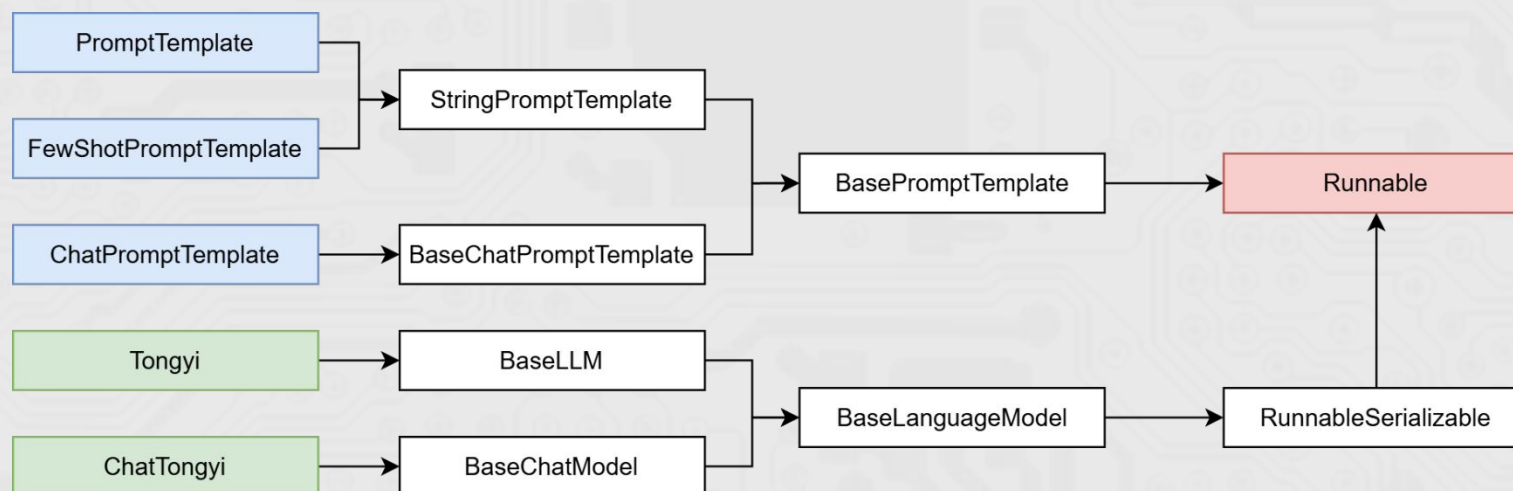
chain链

「**将组件串联，上一个组件的输出作为下一个组件的输入**」是 LangChain 链（尤其是 | 管道链）的核心工作原理，这也是链式调用的核心价值：实现数据的自动化流转与组件的协同工作，如下。

chain = prompt_template / model

核心前提：即Runnable子类对象才能入链（以及Callable、Mapping接口子类对象也可加入（后续了解用的不多））。

我们目前所学习到的组件，均是Runnable接口的子类，如下类的继承关系：



chain链

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.runnables.base import RunnableSerializable
```

```
chat_prompt_template = ChatPromptTemplate.from_messages(
    [
        ("system", "你是一个边塞诗人，可以作诗。"),
        MessagesPlaceholder("history"),
        ("human", "请再来一首唐诗，无需额外输出"),
    ]
)
```

```
history_data = [
    ("human", "你来写一个唐诗"),
    ("ai", "床前明月光，疑是地上霜，举头望明月，低头思故乡"),
    ("human", "好诗再来一个"),
    ("ai", "锄禾日当午，汗滴禾下锄，谁知盘中餐，粒粒皆辛苦"),
]
```

```
model = ChatTongyi(model="qwen3-max")
```

```
chain: RunnableSerializable = chat_prompt_template | model
print(type(chain))
```

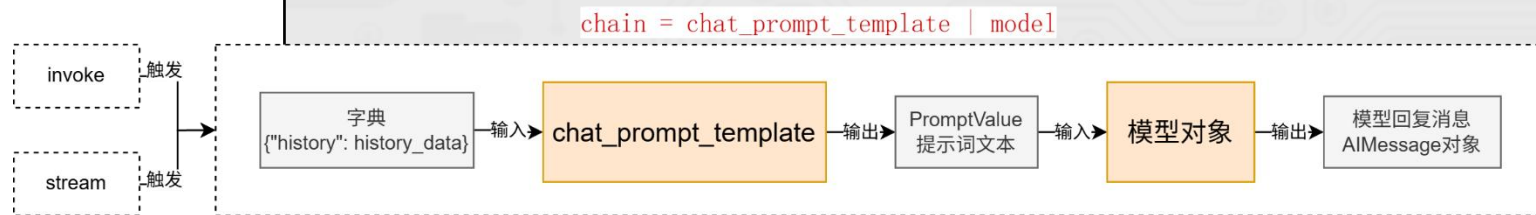
```
# Runnable接口, invoke执行
res = chain.invoke({"history": history_data})
print(res.content)
```

```
# Runnable接口, stream执行
for chunk in chain.stream({"history": history_data}):
    print(chunk.content, end="", flush=True)
```

- 通过|链接提示词模板对象和模型对象
- 返回值chain对象是RunnableSerializable对象
 - 是Runnable接口的直接子类
 - 也是绝大多数组件的父类
- 通过invoke或stream进行阻塞执行或流式执行

组成的链在执行上有：上一个组件的输出作为下一个组件的输入的特性。

所以有如下执行流程：



总结

LangChain中链是一种将各个组件串联在一起，按顺序执行，前一个组件的输出作为下一个组件的输入。

- 可以通过“|”符号来让各个组件形成链
- 成链的各个组件，需是Runnable接口的子类
- 形成的链是RunnableSerializable对象（Runnable接口子类）
- 可通过链调用invoke或stream触发整个链条的执行

```
chain = chat_prompt_template | model
```



AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

|运算符的重写

| 运算符重载

前文代码中: `chain = chat_prompt_template / model`

在语法上使用了|运算符的重写

在 Python 中, 运算符 (如 +、|) 的行为由类的魔法方法决定。例如:

- `a + b` 本质调用的是 `a.__add__(b)`
- `a | b` 本质调用的是 `a.__or__(b)`

只需要自行实现类的 `__or__` 方法, 即可对|符号的功能进行重写。

示例:

- 让 `a|b|c` 的代码得到一个自定义的类对象(类似列表即`[a, b, c]`)
- 调用`run`方法依次输出`a`、`b`、`c`
- 我们需要重写 | 即 `__or__` 方法

| 运算符重载

```
class Test(object):
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Test({self.name})"

    def __or__(self, other):
        return MySequence(self, other)
```

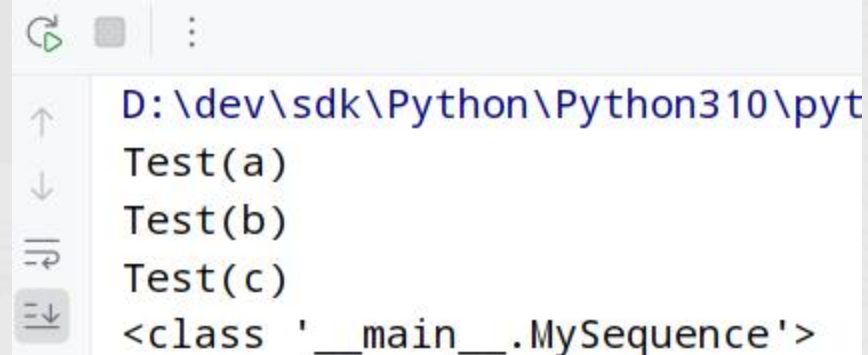
```
class MySequence(object):
    def __init__(self, *args):
        self.sequence = []
        for arg in args:
            self.sequence.append(arg)

    def __or__(self, other):
        self.sequence.append(other)
        return self

    def run(self):
        for arg in self.sequence:
            print(arg)
```

```
if __name__ == "__main__":
    a = Test("a")
    b = Test("b")
    c = Test("c")

    d = a | b | c
    d.run()
    print(type(d))
```



```
D:\dev\sdk\Python\Python310\pyt
Test(a)
Test(b)
Test(c)
<class '__main__.MySequence'>
```

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

StrOutputParser解析器

StrOutputParser字符串输出解析器

有如下代码，想要以第一次模型的输出结果，第二次去询问模型：

```
from langchain_core.prompts import PromptTemplate
from langchain_community.chat_models.tongyi import ChatTongyi

model = ChatTongyi(model="qwen3-max")

prompt = PromptTemplate.from_template(
    "我邻居姓: {lastname}, 刚生了{gender}, 请起名, 仅告知名字无需其它内容"
)

chain = prompt | model | model
res = chain.invoke({"lastname": "张", "gender": "女儿"})
print(res.content)
```

- 链的构建完全符合要求（参与的组件）
- 但是运行报错（`ValueError: Invalid input type <class 'langchain_core.messages.ai.AIMessage'>. Must be a PromptValue, str, or list of BaseMessages.`）

StrOutputParser字符串输出解析器

```
chain = prompt | model | model
```

错误的主要原因是：

- prompt的结果是PromptValue类型，输入给了model
- model的输出结果是：**AIMessage**

模型（ChatTongyi）源码中关于invoke方法明确指定了input的类型：

```
@override
def invoke(
    self,
    input: LanguageModelInput,
    config: RunnableConfig | None = None,
    *,
    stop: list[str] | None = None,
    **kwargs: Any,
) -> AIMessage:
```

需要做类型转换
可以借助LangChain内置的解析器

- StrOutputParser字符串输出解析器

```
LanguageModelInput = PromptValue | str | Sequence[MessageLikeRepresentation]
"""Input to a language model."""
```

StrOutputParser字符串输出解析器

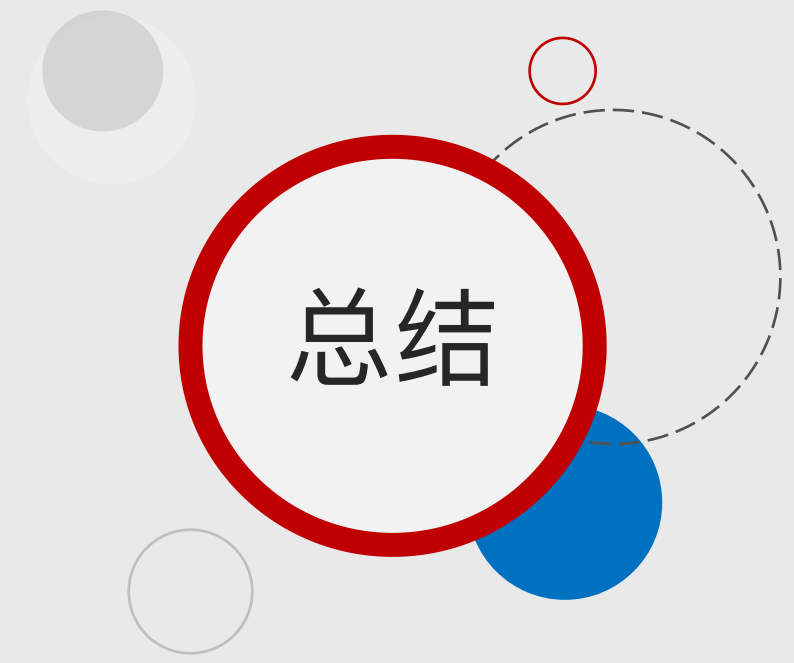
StrOutputParser是LangChain内置的简单字符串解析器

- 可以将AIMessage解析为简单的字符串，符合了模型invoke方法要求（可传入字符串，不接收AIMessage类型）
- 是Runnable接口的子类（可以加入链）

```
parser = StrOutputParser()
chain = prompt | model | parser | model
```

```
@override
def invoke(
    self,
    input: LanguageModelInput,
    config: RunnableConfig | None = None,
    *,
    stop: list[str] | None = None,
    **kwargs: Any,
) -> AIMessage:
```

```
LanguageModelInput = PromptValue | str | Sequence[MessageLikeRepresentation]
"""Input to a language model."""
```



总结

StrOutputParser是LangChain内置的简单字符串解析器。

- 可以将AIMessage类型转换为基础字符串
- 可以加入chain作为组件存在（Runnable接口子类）

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

Runnable接口

Runnable接口

LangChain 中的绝大多数核心组件都继承了 Runnable 抽象基类（位于 langchain_core.runnables.base）。

代码：

`chain = prompt / model`

chain变量是RunnableSequence（RunnableSerializable子类）类型

而得到这个类型的原因就是Runnable基类内部对`__or__`魔术方法的改写。

同时，在后面继续使用添加新的组件，依旧会得到RunnableSequence，这就是链的基础架构。

```
def __or__(
    self,
    other: Runnable[Any, Other]
    | Callable[[Iterator[Any]], Iterator[Other]]
    | Callable[[AsyncIterator[Any]], AsyncIterator[Other]]
    | Callable[[Any], Other]
    | Mapping[str, Runnable[Any, Other] | Callable[[Any], Other] | Any],
) -> RunnableSerializable[Input, Other]:
    """Runnable "or" operator.

    Compose this `Runnable` with another object to create a
    `RunnableSequence`.

    Args:
        other: Another `Runnable` or a `Runnable`-like object.

    Returns:
        A new `Runnable`.

    """
    return RunnableSequence(*steps: self, coerce_to_runnable(other))
```

RunnableSequence对象

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

JsonOutputParser&多模型执行链

JsonOutputParser

```
chain = prompt | model | parser | model | parser
```

在前面我们完成了这样的需求去构建多模型链，不过这种做法并不标准，因为：

上一个模型的输出，没有被处理就输入下一个模型。

正常情况下我们应该有如下处理逻辑：

```
invoke | stream 初始输入 → 提示词模板 → 模型 → 数据处理 → 提示词模板 → 模型 → 解析器 → 结果
```

即：

- 上一个模型的输出结果，应该作为提示词模版的输入，构建下一个提示词，用来二次调用模型。



JsonOutputParser

根据输出和输入的要求：

invoke | stream 初始输入 → 提示词模板 → 模型 → **数据处理** → **提示词模板** → 模型 → 解析器 → 结果

- 模型的输出为：AIMessage类对象
- 提示词模板要求输入如右侧代码：

```
def invoke(  
    self, input: dict, config: RunnableConfig | None = None, **kwargs: Any  
) -> PromptValue:
```

所以，我们需要完成：

将模型输出的AIMessage → 转为字典 → 注入第二个提示词模板中，形成新的提示词（PromptValue对象）



StrOutputParser不满足 (AIMessage → Str)

更换JsonOutputParser (AIMessage → Dict(JSON))



JsonOutputParser完成多模型链

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_community.chat_models.tongyi import ChatTongyi

str_parser = StrOutputParser()
json_parser = JsonOutputParser()

model = ChatTongyi(model="qwen3-max")

first_prompt = PromptTemplate.from_template(
    "我邻居姓: {lastname}, 刚生了{gender}, 请起名, 并封装到JSON格式返回给我, "
    "要求key是name, value就是起的名字。请严格遵守格式要求"
)

second_prompt = PromptTemplate.from_template(
    "姓名{name}, 请帮我解析含义。"
)

chain = first_prompt | model | json_parser | second_prompt | model | str_parser

res: str = chain.invoke({"lastname": "张", "gender": "女儿"})
print(res)
print(type(res))
```



总结

在构建链的时候要注意整体兼容性，注意前后组件的输入和输出要求。

- 模型输入：PromptValue或字符串或序列（BaseMessage、list、tuple、str、dict）。
- 模型输出：AIMessage
- 提示词模板输入：要求是字典
- 提示词模板输出：PromptValue对象
- StrOutputParser：AIMessage输入、str输出
- JsonOutputParser:AIMessage输入、dict输出



AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

RunnableLambda&函数加入链

RunnableLambda

```
chain = first_prompt | model | json_parser | second_prompt | model | str_parser
```

前文我们根据JsonOutputParser完成了多模型执行链条的构建。

- 除了JsonOutputParser这类固定功能的解析器之外
- 我们也可以自己编写Lambda匿名函数来完成自定义逻辑的数据转换，想怎么转换就怎么转换，更自由。

想要完成这个功能，可以基于RunnableLambda类实现。

RunnableLambda类是LangChain内置的，将普通函数等转换为Runnable接口实例，方便自定义函数加入chain。

语法：

RunnableLambda(函数对象或lambda匿名函数)



RunnableLambda

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
from langchain_core.prompts import PromptTemplate
from langchain_community.chat_models.tongyi import ChatTongyi

str_parser = StrOutputParser()
my_func = RunnableLambda(lambda ai_msg: {"name": ai_msg.content})

model = ChatTongyi(model="qwen3-max")

first_prompt = PromptTemplate.from_template(
    "我邻居姓: {lastname}, 刚生了{gender}, 请起名, 仅告知我名字, 不要额外信息"
)

second_prompt = PromptTemplate.from_template(
    "姓名{name}, 请帮我解析含义。"
)

chain = first_prompt | model | my_func | second_prompt | model | str_parser

res: str = chain.invoke({"lastname": "张", "gender": "女儿"})
print(res)
print(type(res))
```



函数直接入链

```
chain = first_prompt | model | (lambda ai_msg: {"name": ai_msg.content}) | second_prompt | model | str_parser
```

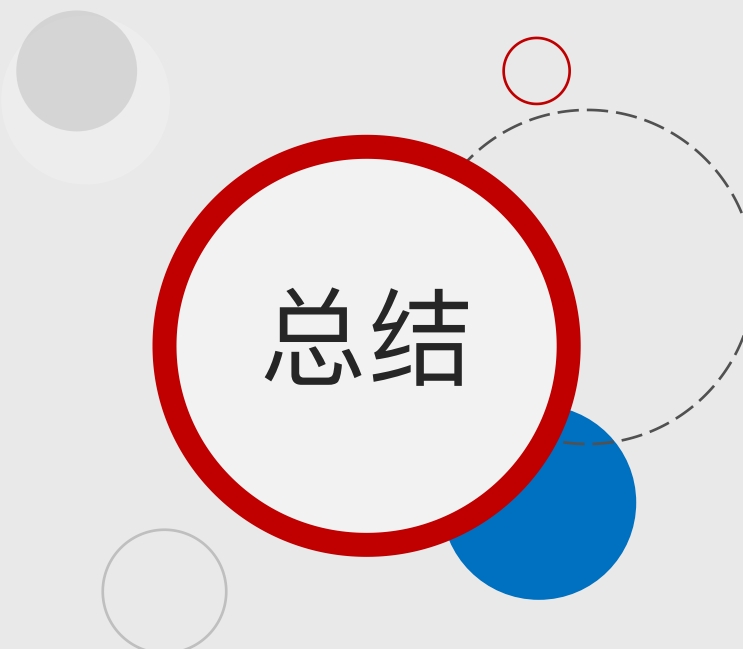
跳过RunnableLambda类，直接让函数加入链也是可以的。
因为Runnable接口类在实现__or__的时候，支持Callable接口的实例。

- 函数就是Callable接口的实例

```
def __or__(  
    self,  
    other: Runnable[Any, Other]  
    | Callable[[Iterator[Any]], Iterator[Other]]  
    | Callable[[AsyncIterator[Any]], AsyncIterator[Other]]  
    | Callable[[Any], Other]  
    | Mapping[str, Runnable[Any, Other] | Callable[[Any], Other] | Any],  
    ) -> RunnableSerializable[Input, Other]:
```

如上代码示例，|符号（底层是调用__or__）组链，是支持函数加入的。
其本质是将函数自动转换为RunnableLambda





总结

如果像要在链中加入自定义函数，可以选择：

- 将函数封装入RunnableLambda类对象，其是Runnable接口实例，可以直接入链
- 直接将函数入链，函数会自动转换为RunnableLambda对象

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

Memory 临时会话记忆

InMemoryChatMessageHistory

临时记忆

如果想要封装历史记录，除了自行维护历史消息外，也可以借助LangChain内置的历史记录附加功能。

LangChain提供了History功能，帮助模型在有历史记忆的情况下回答。

- 基于RunnableWithMessageHistory在原有链的基础上创建带有历史记录功能的新链（新Runnable实例）
- 基于InMemoryChatMessageHistory为历史记录提供内存存储（临时用）

```
from langchain_core.runnables.history import RunnableWithMessageHistory

# 通过RunnableWithMessageHistory获取一个新的带有历史记录功能的chain
conversation_chain = RunnableWithMessageHistory(
    some_chain,          # 被附加历史消息的Runnable，通常是chain
    None,                # 获取指定会话ID的历史会话的函数
    input_messages_key="input",      # 声明用户输入消息在模板中的占位符
    history_messages_key="chat_history" # 声明历史消息在模板中的占位符
)
```

```
# 获取指定会话ID的历史会话记录函数
chat_history_store = {} # 存放多个会话ID所对应的历史会话记录
# 函数传入为会话ID（字符串类型）
# 函数要求返回BaseChatMessageHistory的子类
# BaseChatMessageHistory类专用于存放某个会话的历史记录
# InMemoryChatMessageHistory是官方自带的基于内存存放历史记录的类
def get_history(session_id):
    if session_id not in chat_history_store:
        # 返回一个新的实例
        chat_history_store[session_id] = InMemoryChatMessageHistory()
    return chat_history_store[session_id]
```



完整代码：

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables.history import RunnableWithMessageHistory

def print_prompt(full_prompt):
    print("="*20, full_prompt.to_string(), "="*20)
    return full_prompt

model = ChatTongyi(model="qwen3-max")
prompt = PromptTemplate.from_template(
    "你需要根据对话历史回应用户问题。对话历史: {chat_history}。"
    "用户当前输入: {input}, 请给出回应"
)

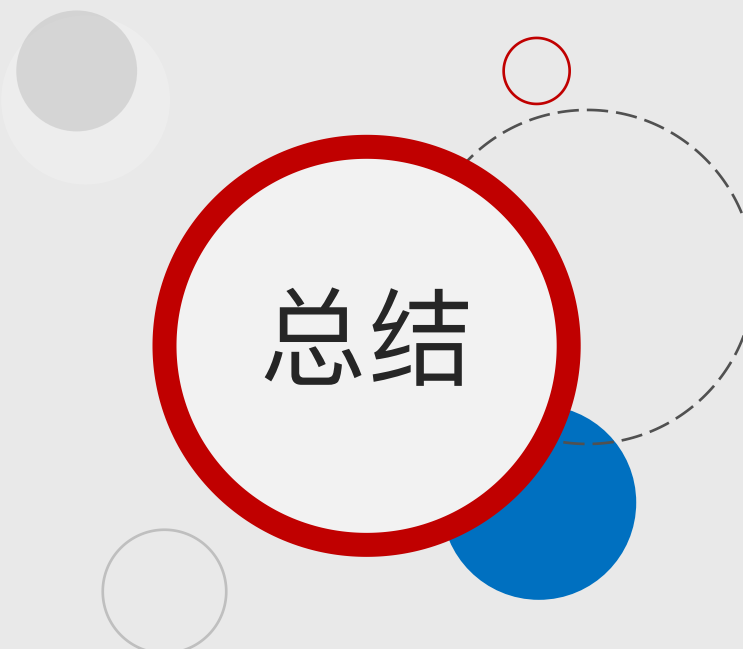
base_chain = prompt | print_prompt | model | StrOutputParser()
```

```
chat_history_store = {} # 存放多个会话ID所对应的历史会话记录
def get_history(session_id):
    if session_id not in chat_history_store:
        # 存入新的实例
        chat_history_store[session_id] = InMemoryChatMessageHistory()
    return chat_history_store[session_id]

# 通过RunnableWithMessageHistory获取一个新的带有历史记录功能的chain
conversation_chain = RunnableWithMessageHistory(
    base_chain, # 被附加历史消息的Runnable, 通常是chain
    get_history, # 获取历史会话的函数
    input_messages_key="input", # 声明用户输入消息在模板中的占位符
    history_messages_key="chat_history" # 声明历史消息在模板中的占位符
)

if __name__ == '__main__':
    # 如下固定格式, 配置当前会话的ID
    session_config = {"configurable": {"session_id": "user_001"}}

    print(conversation_chain.invoke({"input": "小明有一只猫"}, session_config))
    print(conversation_chain.invoke({"input": "小刚有两只狗"}, session_config))
    print(conversation_chain.invoke({"input": "共有几只宠物?"}, session_config))
```



总结

RunnableWithMessageHistory是LangChain内Runnable接口的实现，主要用于：

- 创建一个带有历史记忆功能的Runnable实例（链）

它在创建的时候需要提供一个BaseChatMessageHistory的具体实现（用来存储历史消息）

- InMemoryChatMessageHistory可以实现在内存中存储历史

额外的，如果想要在invoke或stream执行链的同时，将提示词print出来，可以在链中加入自定义函数实现。

- 注意：函数的输入应原封不动返回出去，避免破坏原有业务，仅在return之前，print所需信息即可。

AI 大模型

RAG与智能体开发

Part : LangChain RAG开发

LangChain组件

Memory 长期会话记忆

自实现FileChatMessageHistory

memory 长期会话记忆

使用InMemoryChatMessageHistory仅可以在内存中临时存储会话记忆，一旦程序退出，则记忆丢失。

InMemoryChatMessageHistory 类继承自 BaseChatMessageHistory

```
```python
import json
import os
from langchain_core.messages import messages_from_dict, message_to_dict

class FileChatMessageHistory(BaseChatMessageHistory):
 storage_path: str
 session_id: str

 @property
 def messages(self) -> list[BaseMessage]:
 try:
 with open(
 os.path.join(self.storage_path, self.session_id),
 "r",
 encoding="utf-8",
) as f:
 messages_data = json.load(f)
 return messages_from_dict(messages_data)
 except FileNotFoundError:
 return []

 def add_messages(self, messages: Sequence[BaseMessage]) -> None:
 all_messages = list(self.messages) # Existing messages
```

在官方注释中给出了相关实现的指南，并给出了基于文件的历史消息存储示例代码。

我们可以自行实现一个基于Json格式和本地文件的会话数据保存。

# memory 长期会话记忆

FileChatMessageHistory类实现，核心思路：

- 基于文件存储会话记录，以session\_id为文件名，不同session\_id有不同文件存储消息

继承BaseChatMessageHistory实现如下3个方法：

- add\_messages:同步模式，添加消息
- messages:同步模式，获取消息
- clear: 同步模式，清除消息

如右侧代码，官方在BaseChatMessageHistory类的注释中提供了一个基于文件存储的示例代码。

```
1 import json, os
2 from langchain_core.messages import messages_from_dict, message_to_dict
3
4 class FileChatMessageHistory(BaseChatMessageHistory):
5 storage_path: str
6 session_id: str
7 @property
8 def messages(self) -> list[BaseMessage]:
9 try:
10 with open(
11 os.path.join(self.storage_path, self.session_id),
12 "r",
13 encoding="utf-8",
14) as f:
15 messages_data = json.load(f)
16 return messages_from_dict(messages_data)
17 except FileNotFoundError:
18 return []
19
20 def add_messages(self, messages: Sequence[BaseMessage]) -> None:
21 all_messages = list(self.messages) # Existing messages
22 all_messages.extend(messages) # Add new messages
23
24 serialized = [message_to_dict(message) for message in all_messages]
25 file_path = os.path.join(self.storage_path, self.session_id)
26 os.makedirs(os.path.dirname(file_path), exist_ok=True)
27 with open(file_path, "w", encoding="utf-8") as f:
28 json.dump(serialized, f)
29
30 def clear(self) -> None:
31 file_path = os.path.join(self.storage_path, self.session_id)
32 os.makedirs(os.path.dirname(file_path), exist_ok=True)
33 with open(file_path, "w", encoding="utf-8") as f:
34 json.dump([], f)
```

# memory 长期会话记忆

## 其余核心代码

```
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import BaseChatMessageHistory, BaseMessage
from langchain_core.output_parsers import StrOutputParser
from langchain_core.messages import messages_from_dict, message_to_dict
from langchain_community.chat_models.tongyi import ChatTongyi
from typing import Sequence, List
import json
```

```
----- 核心业务逻辑 -----
1. 初始化通义千问模型
llm = ChatTongyi(model="qwen3-max")

2. 定义提示词模板（包含对话历史和用户输入）
prompt = PromptTemplate.from_template("""你是一个贴心的助手，需要根据对话历史回应用户的问题。
对话历史: {chat_history}
用户当前输入: {input}
你的回应: """)

3. 构建基础链（提示词 -> 模型 -> 输出解析）
base_chain = prompt | llm | StrOutputParser()

4. 定义会话历史获取函数（为每个会话创建独立的文件存储）
def get_message_history(session_id: str) -> BaseChatMessageHistory:
 """
 根据会话ID获取对应的对话历史存储实例
 :param session_id: 会话唯一ID
 :return: FileChatMessageHistory 实例
 """
 return FileChatMessageHistory(session_id=session_id, storage_path="./chat_history")

5. 包装带对话历史的链式调用
conversation_chain = RunnableWithMessageHistory(
 runnable=base_chain,
 get_session_history=get_message_history, # 历史获取函数
 input_messages_key="input", # 用户输入的参数名
 history_messages_key="chat_history", # 对话历史的参数名
)
```

```
----- 测试多轮对话 -----
if __name__ == "__main__":
 # 会话配置（指定会话ID，用于区分不同用户/会话）
 session_config = {"configurable": {"session_id": "user_001"}}

 # 第一轮对话
 response1 = conversation_chain.invoke({"input": "小明有1只猫"}, config=session_config)
 print("第一轮: ", response1)

 # 第二轮对话
 response2 = conversation_chain.invoke({"input": "小刚有2只狗"}, config=session_config)
 print("\n第二轮: ", response2)

 # 第三轮对话（依赖历史上下文）
 response3 = conversation_chain.invoke(
 {"input": "小明和小刚一共有几只宠物?"},
 config=session_config
)
 print("\n第三轮: ", response3)

 # 测试程序重启后读取历史（注释上面的代码，单独运行下面的代码仍能获取历史）
 # response4 = conversation_chain.invoke(
 # {"input": "分别是什么宠物? "},
 # config=session_config
 #)
 # print("\n重启后第四轮: ", response4)
```

# AI 大模型

## RAG与智能体开发

*Part : LangChain RAG开发*

## LangChain组件

Document loaders: 文档加载器

## Document loaders文档加载器

文档加载器提供了一套标准接口，用于将不同来源（如 CSV、PDF 或 JSON等）的数据读取为 LangChain 的文档格式。这确保了无论数据来源如何，都能对其进行一致性处理。

文档加载器（内置或自行实现）需实现BaseLoader接口。

`Class Document`，是LangChain内文档的统一载体，所有文档加载器最终返回此类的实例。

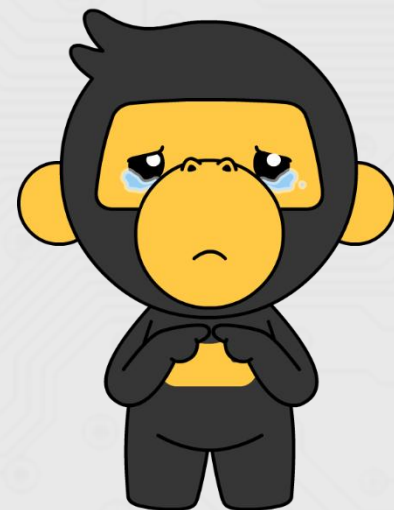
一个基础的Document类实例，基于如下代码创建：

```
from langchain_core.documents import Document

document = Document(
 page_content="Hello, world!", metadata={"source":
"https://example.com"}
)
```

可以看到，Document类其核心记录了：

- `page_content`：文档内容
- `metadata`：文档元数据（字典）



## Document loaders文档加载器

不同的文档加载器可能定义了不同的参数，但是其都实现了统一的接口（方法）。

- `load()`: 一次性加载全部文档
- `lazy_load()`: 延迟流式传输文档，对大型数据集很有用，避免内存溢出。

一个简单的CSVLoader的使用示例如下：

```
from langchain_community.document_loaders.csv_loader
import CSVLoader

loader = CSVLoader(
 ... # 初始化参数
)

一次性加载全部文档
documents = loader.load()

对于大数据集，分段返回文档
for document in loader.lazy_load():
 print(document)
```



# CSVLoader

LangChain内置了许多文档加载器，详细参见官方文档：

[https://docs.langchain.com/oss/python/integrations/document\\_loaders](https://docs.langchain.com/oss/python/integrations/document_loaders)

我们简单的学习如下几个常用的文档加载器：

- CSVLoader
- JSONLoader
- PDFLoader

```
from langchain_community.document_loaders.csv_loader import CSVLoader

loader = CSVLoader(file_path="./xxx.csv")

data = loader.load()

print(data)
```



# CSVLoader

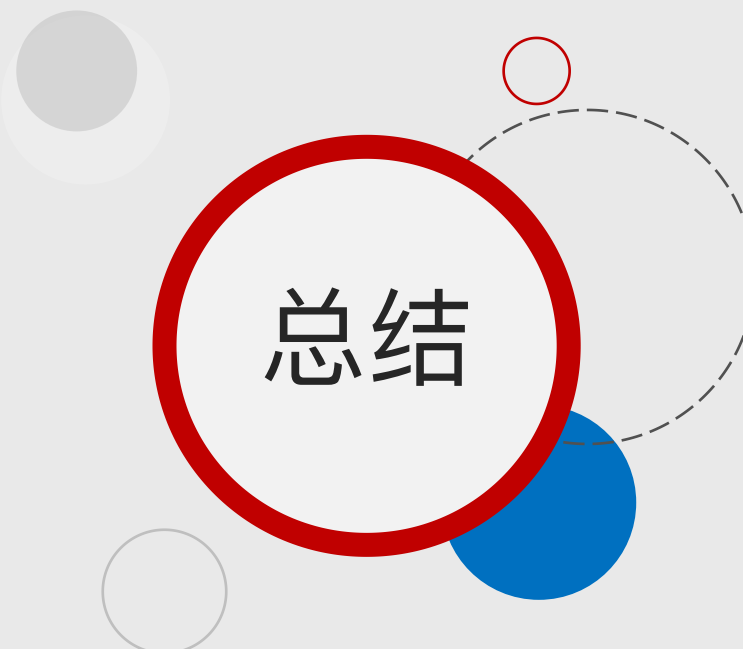
## 自定义CSV文件的解析和加载

```
loader = CSVLoader(
 file_path="./xxx.csv",
 csv_args={
 "delimiter": ",", # 指定分隔符
 "quotechar": '"' # 指定字符串的引号包裹
 # 字段列表（无表头使用，有表头勿用会读取首行做为数据）
 "fieldnames": ["name", "age", "gender"],
 },
)

data = loader.load()

print(data)
```





# 总结

LangChain内置了许多种类的文档加载器

- 文档加载器均继承于BaseLoader类
- 返回Document类型的对象
- load方法一次性批量加载（返回list内含Document对象），如内容过多可能list太大，出现内存溢出问题
- lazy\_load方法会得到生成器对象，可用for循环依次获取单个Document对象，适用于大文档避免内存存不下。

CSVLoader用于加载CSV文件，加载成功得到的即Document对象。

# AI 大模型

## RAG与智能体开发

*Part : LangChain RAG开发*

## LangChain组件

Document loaders: JSONLoader

# JSONLoader

JSONLoader用于将JSON数据加载为Document类型对象。

使用JSONLoader需要额外安装： `pip install jq`

jq是一个跨平台的json解析工具，LangChain底层对JSON的解析就是基于jq工具实现的。  
将JSON数据的信息抽取出来，封装为Document对象，抽取的时候依赖jq\_schema语法。

```
{
 "name": "周杰轮",
 "age": 11,
 "hobby": ["唱", "跳", "RAP"],
 "other": {
 "addr": "深圳",
 "tel": "12332112321"
 }
}
```

- `.`表示整个JSON对象（根）
- `[]`表示数组
- `.name`表示抽取周杰轮
- `.hobby`表示抽取爱好数组
- `.hobby[1]`或`.hobby.[1]`表示抽取跳
- `.other.addr`表示抽取地址深圳

```
[
 {"name": "周杰轮", "age": 11, "gender": "男"},
 {"name": "蔡依临", "age": 12, "gender": "女"},
 {"name": "王力鸿", "age": 11, "gender": "男"}
]
```

- `.[].`得到3个字典
- `.[].name` 表示抽取全部的名字，即得到3个name信息



# JSONLoader

了解jq的基本抽取规则后，即可使用JSONLoader加载JSON文件了。

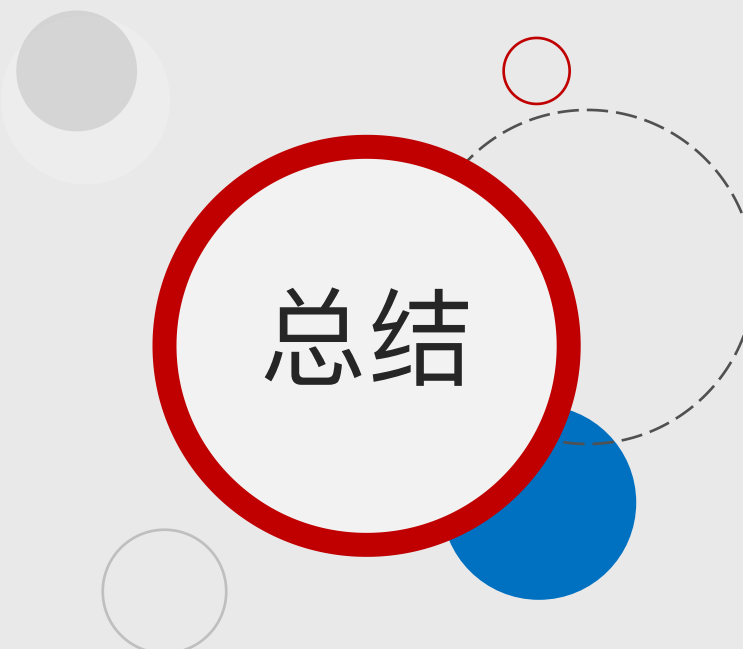
```
from langchain_community.document_loaders import JSONLoader
```

```
loader = JSONLoader(
 file_path="xxx.json", # 文件路径
 jq_schema=".", # jq schema语法
 text_content=False, # 抽取的是否是字符串, 默认True
 json_lines=True, # 是否是JsonLines文件 (每一行都是JSON的文件)
)
```

如下是一个典型的JsonLines文件

```
{"name": "周杰轮", "age": 11, "gender": "男"}
{"name": "蔡依临", "age": 12, "gender": "女"}
{"name": "王力鸿", "age": 11, "gender": "男"}
```





# 总结

JSONLoader依赖jq库，通过pip install jq安装。

- JSONLoader使用jq的解析语法，常见如：
  - .表示根、[]表示数组
  - .name表示从根取name的值
  - .hobby[1]表示取hobby对应数组的第二个元素
  - .[]表示将数组内的每个字典（JSON对象）都取到
  - .[].name表示取数组内每个字典（JSON）对象的name对应的值

JSONLoader初始化有4个主要参数：

- file\_path：文件路径，必填
- jq\_schema：jq解析语法，必填
- text\_content：抽取到的是否是字符串，默认True，非必填
- json\_lines：是否是JsonLines文件，默认False，非必填
  - JsonLines文件：每一行都是一个独立的字典（Json对象）

# AI 大模型

## RAG与智能体开发

*Part : LangChain RAG开发*

## LangChain组件

Document loaders: PyPDFLoader

# PyPDFLoader

LangChain内支持许多PDF的加载器，我们选择其中的PyPDFLoader使用。

PyPDFLoader加载器，依赖PyPDF库，所以，需要安装它：

```
pip install pypdf
```

PyPDFLoader使用还是比较简单的，如下代码即可快速加载PDF中的文字内容了：

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader(
 file_path="", # 文件路径必填
 mode='page', # 读取模式，可选page（按页面划分不同Document）和single（单个Document）
 password='password', # 文件密码
)
```



# AI 大模型

## RAG与智能体开发

*Part : LangChain RAG开发*

### LangChain组件

TextLoader和文档分割器

# TextLoader

除了前文学习的三个Loader以外，还有一个基本的加载器：TextLoader  
作用：读取文本文件（如.txt），将全部内容放入一个Document对象中。

```
from langchain_community.document_loaders
import TextLoader

loader = TextLoader(
 "xxx.txt",
 encoding="utf-8",
)
docs = loader.load()
print(docs)
print(len(docs)) # 结果为1
```



如果文档很大  
加载到一个Document对象中是否不太合适？



# RecursiveCharacterTextSplitter

RecursiveCharacterTextSplitter，递归字符文本分割器，主要用于按自然段落分割大文档。

是LangChain官方推荐的默认字符分割器。

它在保持上下文完整性和控制片段大小之间实现了良好平衡，开箱即用效果佳。

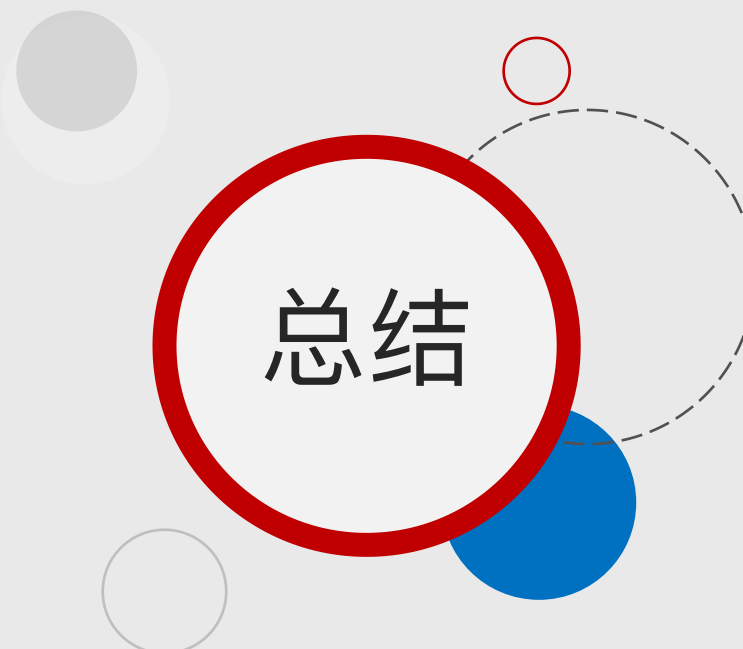
```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = TextLoader(
 "../P3_LangChainRAG开发/data/Python基础语法.txt",
 encoding="utf-8",
)
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(
 chunk_size=500, # 分段的最大字符数
 chunk_overlap=50, # 分段之间允许重叠的字符数
 # 文本分段依据
 separators=["\n\n", "\n", ". ", "! ", "? ", ":", "!", "?", " ", ""],
 # 字符统计依据 (函数)
 length_function=len,
)

split_docs = splitter.split_documents(docs)
```





# 总结

TextLoader是一个简单的加载器，可以加载文本文件内容，返回仅有一个Document对象的list。

RecursiveCharacterTextSplitter递归字符文本分割器，是LangChain官方推荐的默认分割器。

- 基于文本的自然段落分割大文档为小文档
- 可以指定小文档的最大字符数、重叠字符数
- 可以手动指定段落划分的依据（符号）以及字符数量统计函数

# AI 大模型

## RAG与智能体开发

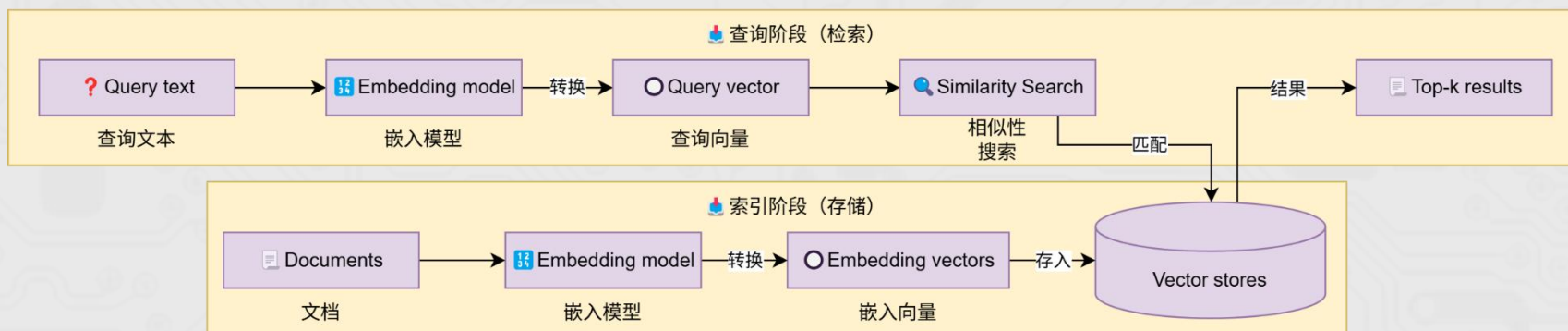
*Part : LangChain RAG开发*

### LangChain组件

Vector stores 向量存储

# Vector stores 向量存储

基于LangChain的向量存储，存储嵌入数据，并执行相似性搜索。



如图，这是一个典型的向量存储应用，也即是典型的RAG流程。

这部分开发主要涉及到：

- 如何文本转向量（前文已经学习）
- 创建向量存储，基于向量存储完成：
  - 存入向量
  - 删除向量
  - 向量检索



LangChain为向量存储提供了统一接口：

- add\_documents
- delete
- similarity\_search



# Vector stores 向量存储

## 内置向量存储的使用

```
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_community.embeddings import DashScopeEmbeddings

vector_store = InMemoryVectorStore(embedding=DashScopeEmbeddings())
添加文档到向量存储, 并指定id
vector_store.add_documents(documents=[doc1, doc2], ids=["id1", "id2"])
删除文档 (通过指定的id删除)
vector_store.delete(ids=["id1"])
相似性搜索
similar_docs = vector_store.similarity_search("your query here", 4)
```

## 外部 (Chroma) 向量存储的使用

```
from langchain_community.embeddings import DashScopeEmbeddings
from langchain_chroma import Chroma

vector_store = Chroma(
 collection_name="example_collection",
 embedding_function=DashScopeEmbeddings(),
 persist_directory="./chroma_langchain_db", # Where to save data locally, remove if not necessary
)
```



# 总结

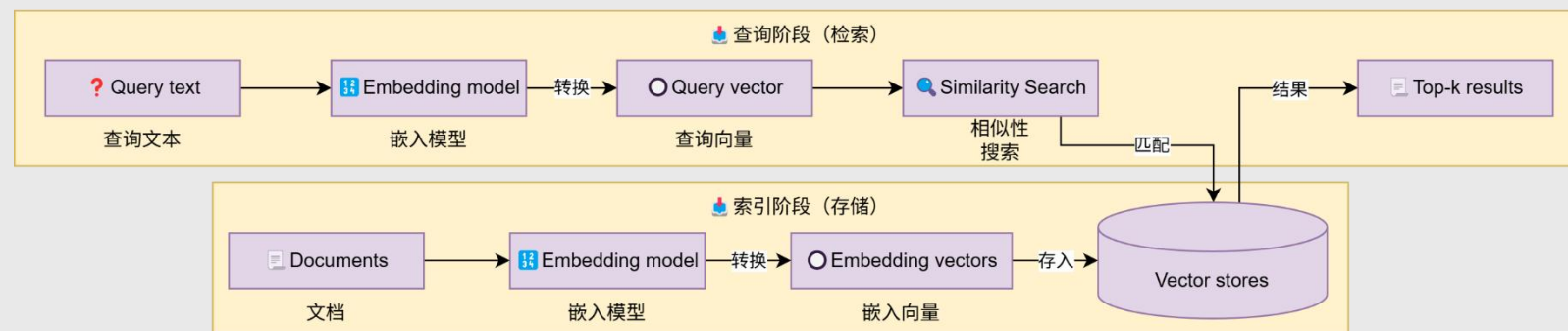
LangChain内提供向量存储功能，可以基于：

- InMemoryVectorStore，完成内存向量存储
- Chroma，外部数据库向量存储

向量存储类均提供3个通用API接口：

- add\_document，添加文档到向量存储
- delete，从向量存储中删除文档
- similarity\_search：相似度搜索

整体向量存储使用流程如下（RAG流程）：



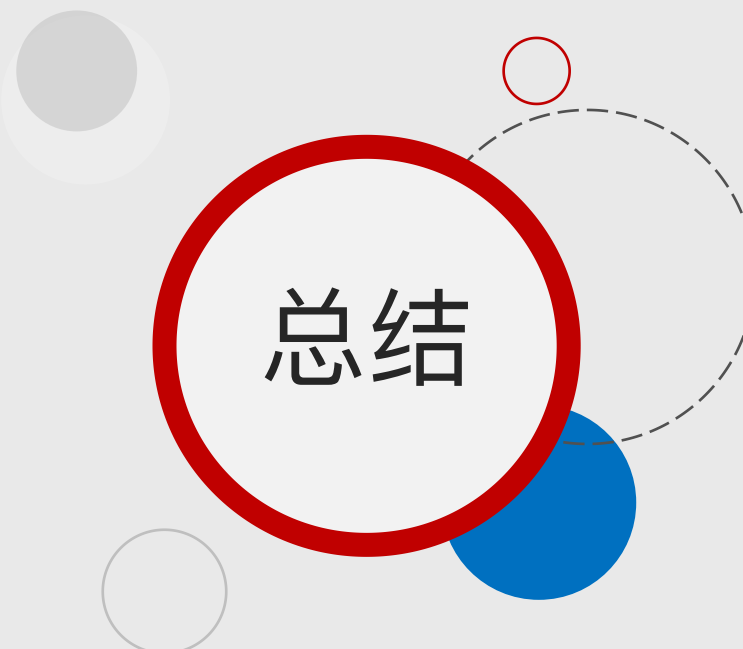
# AI 大模型

## RAG与智能体开发

*Part : LangChain RAG开发*

### LangChain组件

检索向量并构建提示词



# 总结

向量存储的实例，通过`add_texts(list[str])`方法可以快速添加到向量存储中。

流程：

1. 先通过向量存储检索匹配信息
2. 将用户提问和匹配信息一同封装到提示词模板中提问模型

# AI 大模型

## RAG与智能体开发

*Part : LangChain RAG开发*

### LangChain组件

RunnablePassthrough的使用

# RunnablePassthrough的使用

```
27 # 准备一下资料 (向量库的数据)
28 # add_texts 传入一个 list[str]
29 vector_store.add_texts(
30 ["减肥就是要少吃多练", "在减脂期间吃东西很重要,清淡少油控制卡路里摄入并运动起来",
31
32 input_text = "怎么减肥?"
33
34 # 检索向量库
35 result = vector_store.similarity_search(input_text, k=2)
36 reference_text = "["
37 for doc in result:
38 reference_text += doc.page_content
39 reference_text += "]"
40
41 chain = prompt | print_prompt | model | StrOutputParser()
42
43 res = chain.invoke({"input": input_text, "context": reference_text})
44 print(res)
```



让向量检索加入链?

使用RunnablePassthrough类

